

Jessica Ausilia Afriani

Peserta Cyber Security Bootcamp VisioNet Batch XIV

Lab Network Traffic Analysis

Kredensial Melayang

1. Objective

Melakukan analisis network traffic pada file `portal-capture.pcapng` menggunakan Wireshark untuk menemukan informasi yang dikirim melalui komunikasi HTTP. Analisis difokuskan pada proses login portal karyawan, mulai dari identifikasi workstation yang berhasil login, username dan password yang digunakan, hingga session cookie yang dikirim oleh server setelah login berhasil.

Hasil analisis kemudian digunakan untuk menjawab pertanyaan pada service challenge sampai mendapatkan flag.

2. Target

Deskripsi challenge:

Portal karyawan internal masih jalan di HTTP polos. Analisa capture-nya, jawab pertanyaan di service ini.

Artifact yang digunakan:

`portal-capture.pcapng`

3. Tools yang Digunakan

- **Kali Linux**
- **Wireshark** untuk menganalisis network traffic dan file PCAP
- **Netcat (nc)** untuk terhubung ke service challenge
- **cURL** untuk mengecek respons dari service

4. Langkah Pengerjaan

4.1 Spawn Instance

Pertama, membuka halaman challenge:

`http://43.134.2.156:8000/instance/netsec-01-clear-text-credentials`

Kemudian melakukan **Spawn Instance**.

Setelah instance berhasil dibuat, muncul informasi:

Kredensial Melayang

Portal karyawan internal masih jalan di HTTP polos. Analisa capture-nya, jawab pertanyaan di service ini.

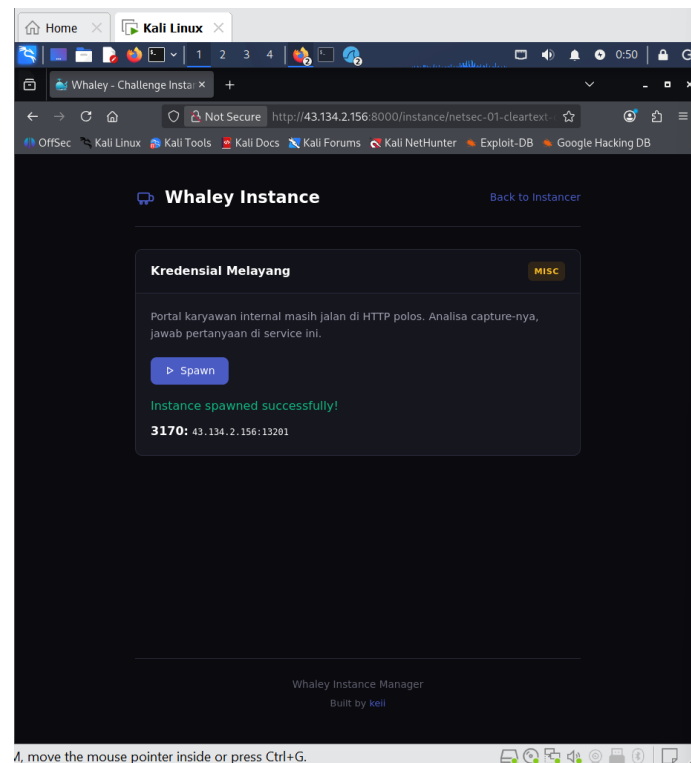
Instance spawned successfully!

3170: 43.134.2.156:13201

Dari informasi tersebut diketahui bahwa service challenge berjalan pada:

IP: 43.134.2.156

Port: 13201



Gambar 1. Hasil Spawn Instance

4.2 Mengecek Koneksi ke Port Service

Sebelum melakukan analisis lebih lanjut, dilakukan pengecekan apakah port 13201 dapat dijangkau dari Kali Linux.

Command yang digunakan:

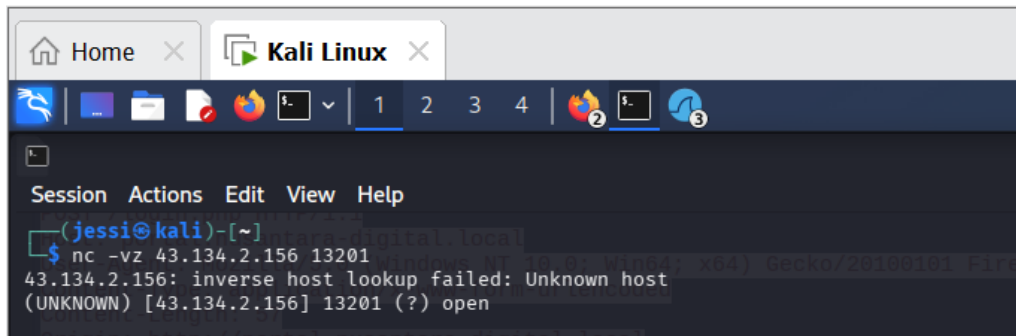
```
nc -vz 43.134.2.156 13201
```

Hasil:

```
43.134.2.156: inverse host lookup failed: Unknown host  
(UNKNOWN) [43.134.2.156] 13201 (?) open
```

Dari hasil tersebut, port 13201 berstatus **open**, sehingga service dapat dijangkau dari Kali Linux.

Pesan `inverse host lookup failed` tidak menjadi masalah karena koneksi ke port tetap berhasil.



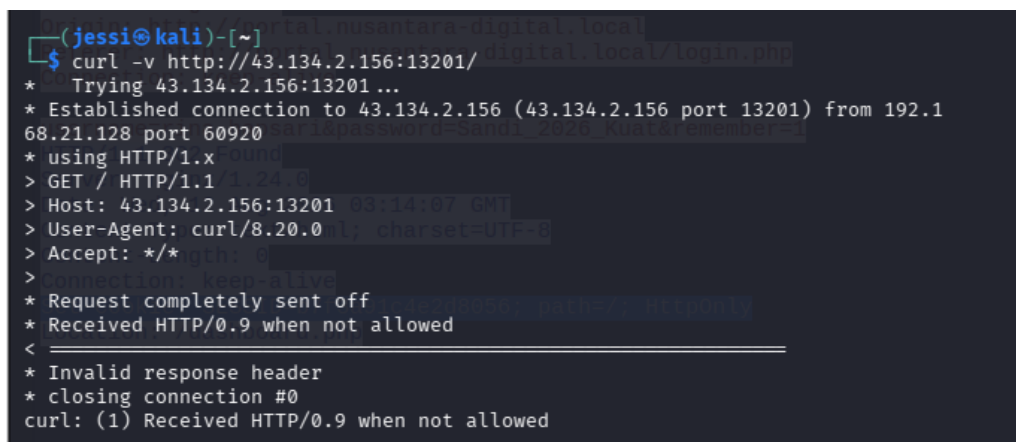
Gambar 2. Hasil Pengecekan Koneksi Port Service

4.3 Mengecek Respons Service Menggunakan cURL

Selanjutnya dilakukan pengecekan menggunakan `curl`:

```
curl -v http://43.134.2.156:13201/
```

Hasil pengecekan menunjukkan port dapat diakses, tetapi service tidak memberikan response HTTP standar yang dapat diproses oleh cURL. Oleh karena itu, analisis challenge dilanjutkan menggunakan `nc`, sesuai karakteristik service yang menyediakan pertanyaan secara interaktif.



Gambar 3. Hasil Pengecekan Respons Service

5. Analisis `portal-capture.pcapng` menggunakan Wireshark

5.1 Membuka File PCAP

Selanjutnya dilakukan analisis terhadap artifact yang diberikan oleh challenge, yaitu:

`portal-capture.pcapng`

File dibuka menggunakan:

Wireshark → File → Open → `portal-capture.pcapng`

5.2 Memfilter Traffic HTTP

Karena pada deskripsi challenge disebutkan bahwa portal karyawan masih menggunakan **HTTP polos**, analisis difokuskan pada traffic HTTP.

Pada bagian **Display Filter** Wireshark digunakan filter:

http

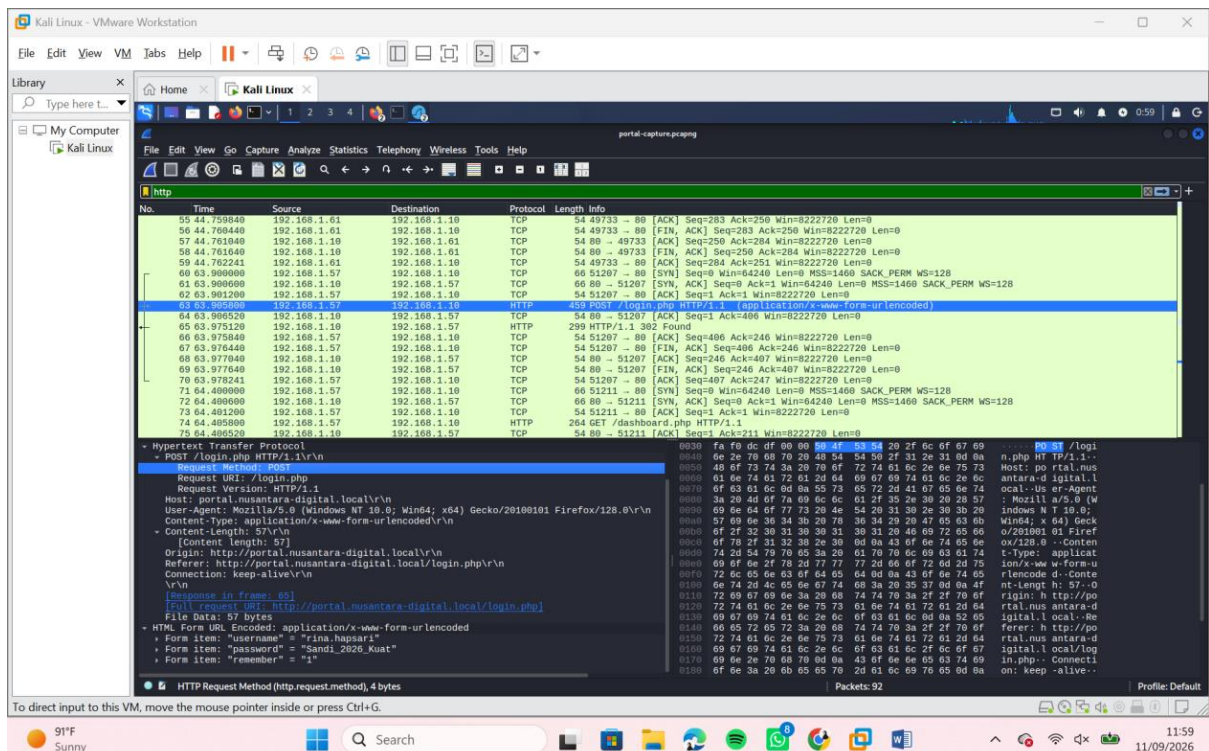
Kemudian menekan **Enter**.

Filter tersebut digunakan untuk menampilkan traffic yang menggunakan protokol HTTP.

Dari hasil filtering kemudian diperhatikan beberapa informasi seperti:

- Time
- Source
- Destination
- Protocol
- Length
- Info

Salah satu hal yang dicari adalah request yang berkaitan dengan proses login.



Gambar 4. Hasil Filter http

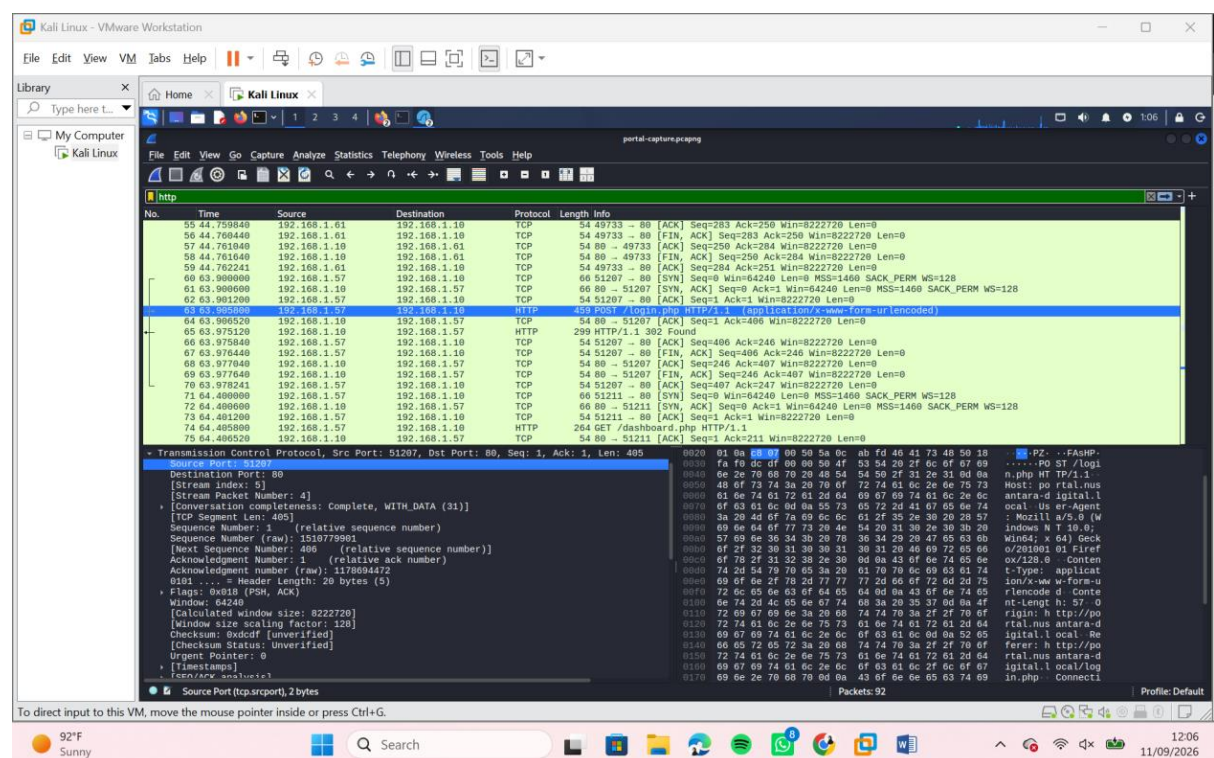
5.3 Mencari Request Login

Setelah traffic HTTP berhasil ditampilkan menggunakan filter `http`, selanjutnya dilakukan pencarian secara manual pada packet yang ditampilkan untuk menemukan request yang berkaitan dengan proses login.

Dari daftar packet HTTP, diperhatikan bagian **Info** untuk mencari request dengan metode `POST` dan URI yang berkaitan dengan proses login, seperti:

`POST /login.php HTTP/1.1`

Setelah request login ditemukan, packet tersebut dipilih untuk dilakukan pemeriksaan lebih lanjut pada bagian detail packet.



Gambar 5. Hasil mencari Request Login menggunakan Filter `http`

5.4 Memeriksa Detail Packet

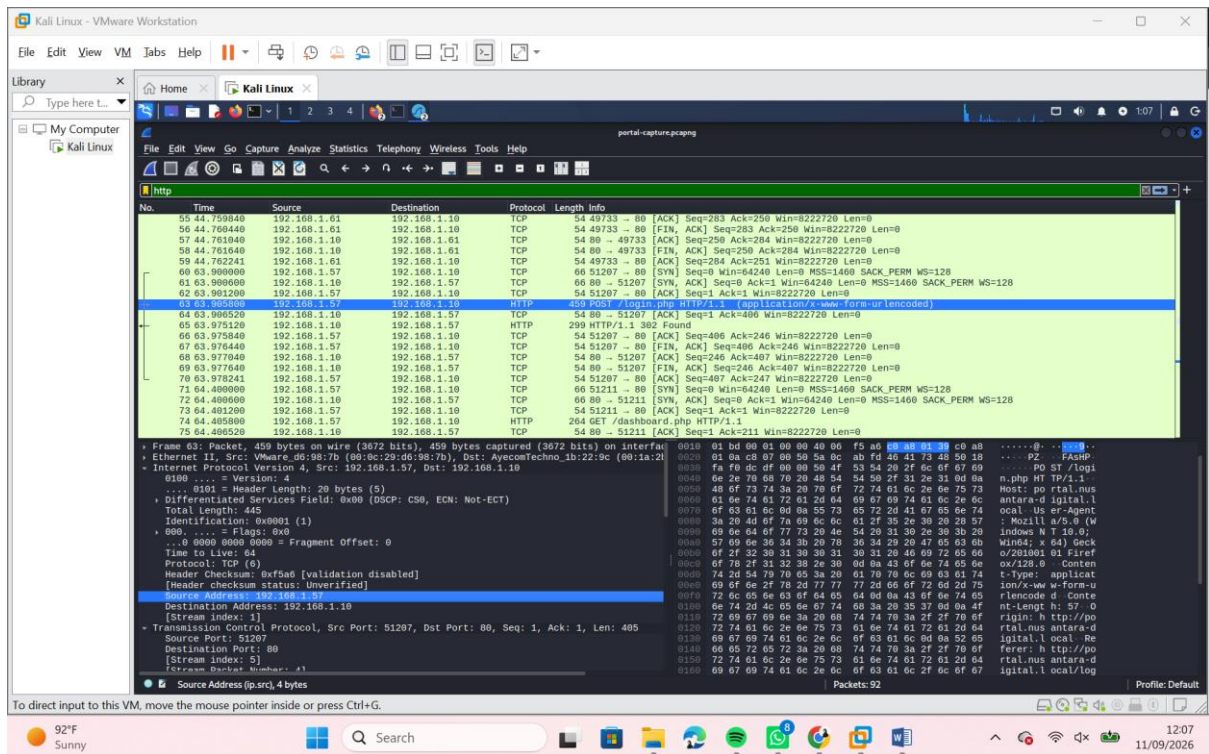
Pada packet yang dipilih, bagian detail packet diperiksa melalui:

Internet Protocol Version 4

Bagian ini digunakan untuk melihat:

Source Address

Destination Address



Gambar 6. Hasil Pengecekan Internet Protocol Version 4

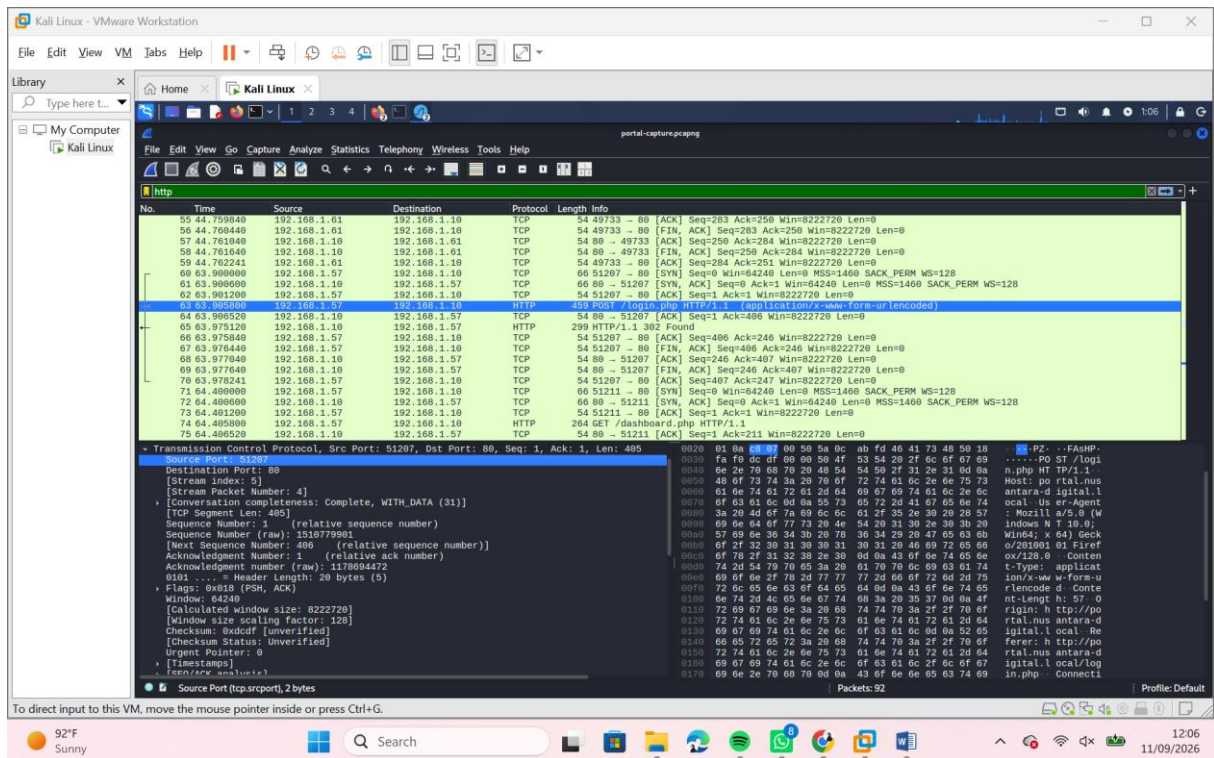
Kemudian bagian:

Transmission Control Protocol

digunakan untuk melihat:

Source Port

Destination Port

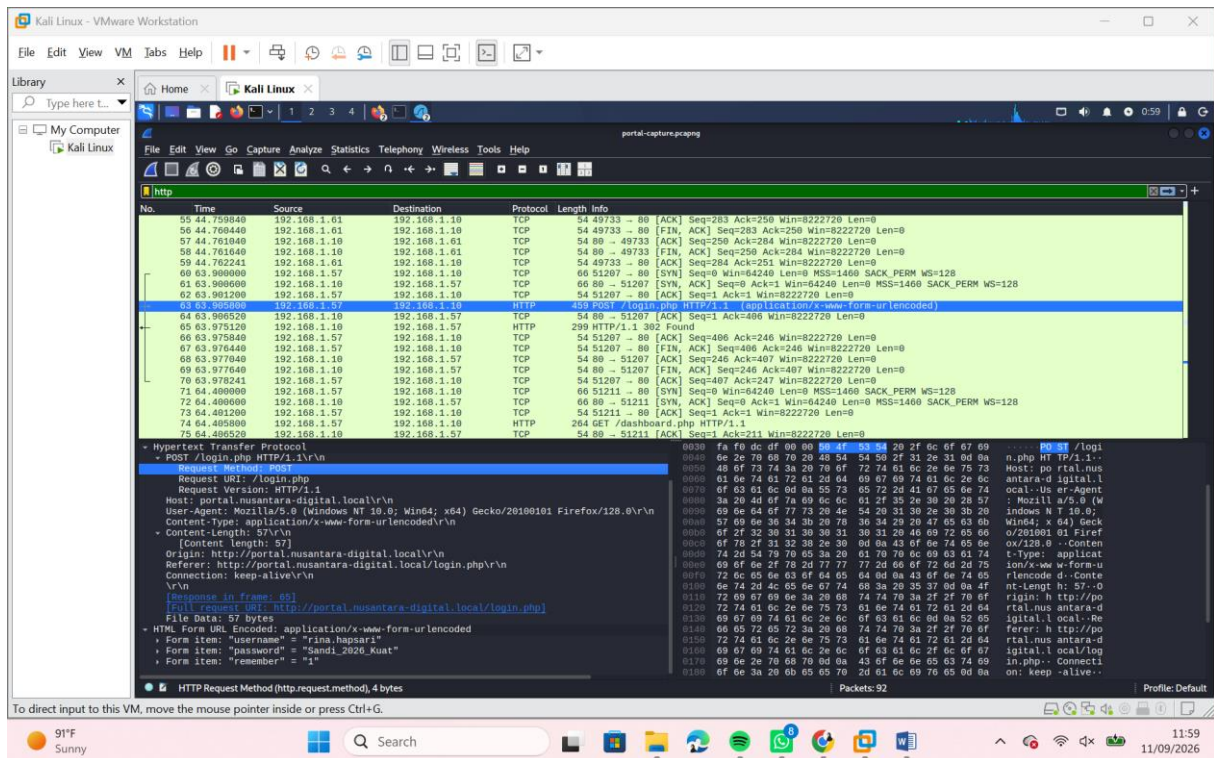


Gambar 7. Hasil Pengecekan Transmission Control Protocol

Dengan demikian, informasi mengenai workstation dan server dapat diketahui dari packet yang dianalisis.

Informasi yang dicatat dari packet antara lain:

Informasi	Hasil
Source IP	192.168.1.57
Destination IP	192.168.1.10
Source Port	51207
Destination Port	80
HTTP Request	POST /login.php
Username	rina.hapsari
Password	Sandi_2026_Kuat



Gambar 8. Hasil Pengecekan Hypertext Transfer Protocol

6. Memastikan Login Berhasil

Tidak semua request login dapat langsung dianggap berhasil. Oleh karena itu, request login yang ditemukan diperiksa bersama response dari server.

Packet login kemudian dapat diperiksa menggunakan command:

Right Click → Follow → HTTP Stream

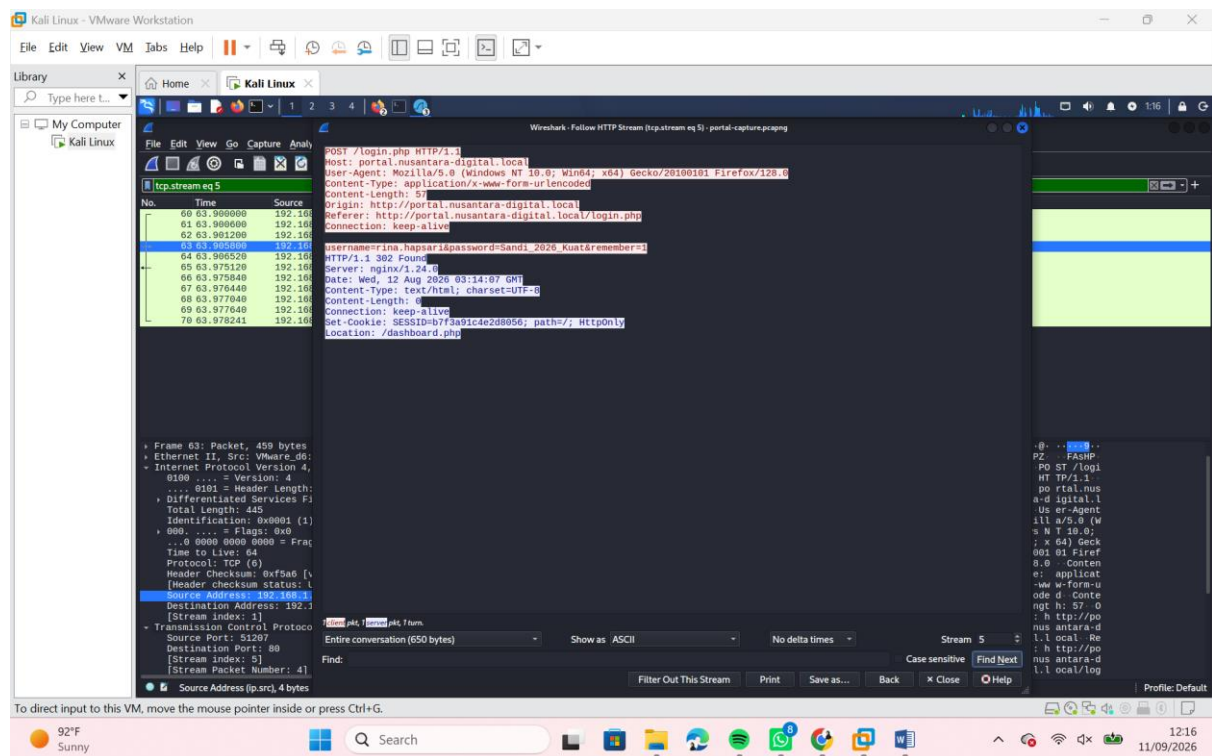
Dengan fitur tersebut, komunikasi antara workstation dan server dapat dilihat dalam satu alur.

Request	Response
Source IP: 192.168.1.57	Status: 302 Found
Destination IP: 192.168.1.10	Set-Cookie: SESSID=B7fa91c4e2d8056
Method: POST	Location: /dashboard.php
Username: rina.hapsari	
Password: Sandi_2026_Kuat	

Analisis:

Status 302 Found menunjukkan server memberikan redirect setelah request login. Header Location: /dashboard.php menunjukkan client diarahkan ke halaman dashboard, sedangkan Set-Cookie menunjukkan server membuat session untuk client. Berdasarkan kombinasi informasi tersebut, request dapat diidentifikasi sebagai login yang berhasil.

Hasil tersebut kemudian digunakan untuk menjawab pertanyaan pada service challenge.



Gambar 9. Hasil Pengecekan Cookie

7. Menjalankan Service Challenge

Setelah melakukan analisis terhadap PCAP, service challenge dijalankan menggunakan:

```
nc 43.134.2.156 13201
```

Kemudian service menampilkan:

```
=====
Kredensial Melayang
=====
Insiden : portal karyawan internal masih HTTP polos
Artifact: portal-capture.pcapng
```

Jawab 4 pertanyaan berikut berdasarkan isi capture.

Service kemudian memberikan empat pertanyaan yang harus dijawab berdasarkan hasil analisis PCAP.

```
(jessi@kali)-[~]
$ nc 43.134.2.156 13201

=====
Kredensial Melayang
=====

Insiden : portal karyawan internal masih HTTP polos
Artifact: portal-capture.pcapng

Jawab 4 pertanyaan berikut berdasarkan isi capture.

Q1) IP address workstation yang login-nya BERHASIL ke portal? > 192.168.1.57
[+] Benar.

Q2) Username yang dipakai pada login berhasil itu? > rina.hapsari
[+] Benar.

Q3) Password akun tersebut (persis, case-sensitive)? > Sandi_2026_Kuat
[+] Benar.

Q4) Nilai cookie sesi yang dikirim server setelah login berhasil? > b7f3a91c4e2d8056
[+] Benar.

=====
Semua jawaban benar.
Flag: FLAG{cl34rt3xt_http_cr3d3nt14l_sn1ff3d}
=====
```

Gambar 10. Hasil menjalankan Service Challenge

8. Hasil Akhir

Setelah seluruh pertanyaan dijawab dengan benar, service memberikan hasil:

```
=====
Semua jawaban benar.
Flag: FLAG{cl34rt3xt_http_cr3d3nt14l_sn1ff3d}
=====
```

Flag yang diperoleh:

FLAG{cl34rt3xt_http_cr3d3nt14l_sn1ff3d}

9. Conclusion

Dari lab **Kredensial Melayang**, saya melakukan analisis terhadap file `portal-capture.pcapng` menggunakan Wireshark untuk melihat komunikasi HTTP antara workstation dan portal karyawan.

Dari traffic tersebut, saya dapat mengidentifikasi workstation yang berhasil melakukan login, username dan password yang digunakan, serta session cookie yang dikirim server setelah login berhasil. Hasil analisis kemudian digunakan untuk menjawab empat pertanyaan pada service challenge.

Challenge ini menunjukkan 2 risiko dari penggunaan HTTP tanpa enkripsi:

- **Credential exposure:** username dan password dapat dibaca dari HTTP plaintext.
- **Session exposure:** session cookie juga dapat terlihat sehingga berpotensi digunakan untuk mengambil alih session jika tidak ada perlindungan tambahan.

Hal tersebut sesuai dengan konsep “Kredensial Melayang”, yaitu kredensial dikirim melalui network traffic tanpa enkripsi sehingga dapat terlihat dan ditemukan melalui analisis packet capture.

Dari seluruh proses tersebut, flag yang berhasil diperoleh adalah:

```
FLAG{cl34rt3xt_http_cr3d3nt14l_sn1ff3d}
```

Siapa yang Ngetok Pintu

1. Objective

Menganalisis network traffic pada capture `recon-capture.pcapng` untuk mengidentifikasi aktivitas scanning pada segmen `192.168.10.0/24`, meliputi IP address penyerang, host yang menjadi target, teknik scanning, port yang terbuka, dan jumlah port unik yang diprobe.

2. Target

Artifact yang digunakan:

- **Artifact:** `recon-capture.pcapng`
- **Target:** `192.168.10.0/24`

3. Tools yang Digunakan

- Wireshark untuk menganalisis network traffic dan file PCAP
- Tshark untuk menganalisis traffic DNS dan mengambil field dari packet capture

4. Langkah Pengerjaan

4.1 Spawn Instance

Pertama, membuka halaman challenge:

<http://43.134.2.156:8000/instance/netsec-02-recon-footprint>

Kemudian melakukan **Spawn Instance**.

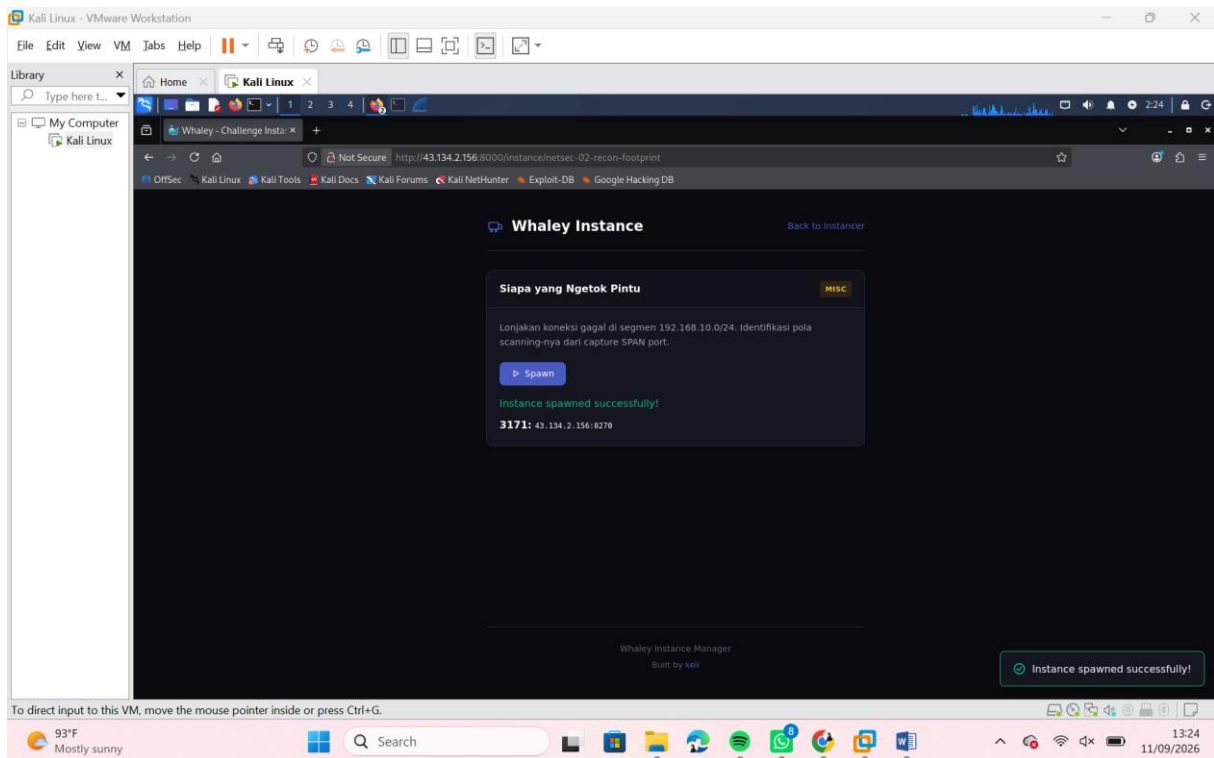
Setelah instance berhasil dibuat, muncul informasi:

```
Siapa yang Ngetok Pintu
Lonjakan koneksi gagal di segmen 192.168.10.0/24, Identifikasi pola scanning-nya dari capture SPAN port.
```

```
Instance spawned successfully!
3171 43.134.2.156:8270
```

Dari informasi tersebut diketahui bahwa service challenge berjalan pada:

```
IP: 43.134.2.156
Port: 8270
```



Gambar 1. Hasil Spawn Instance

4.2 Mengecek Koneksi ke Port Service

Sebelum melakukan analisis lebih lanjut, dilakukan pengecekan apakah port 8270 dapat dijangkau dari Kali Linux.

Command yang digunakan:

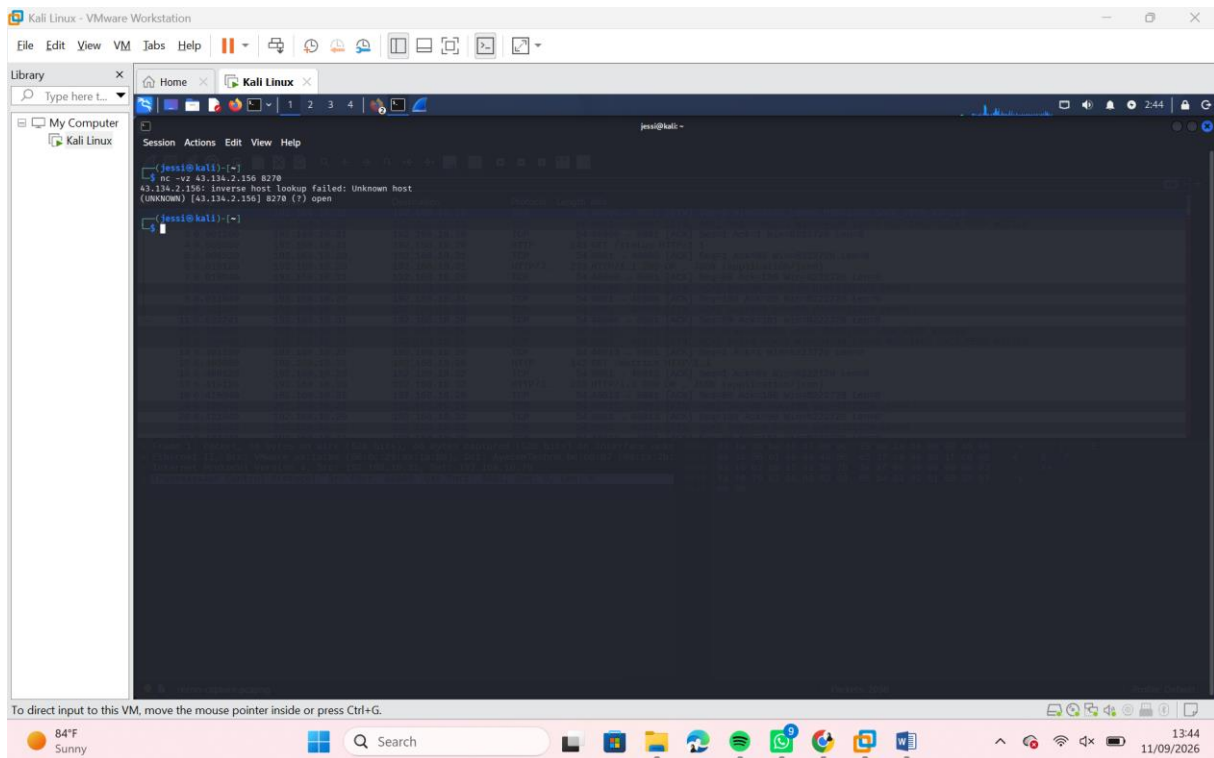
```
nc -vz 43.134.2.156 8270
```

Hasil:

```
43.134.2.156: inverse host lookup failed: Unknown host  
(UNKNOWN) [43.134.2.156] 8270 (?) open
```

Dari hasil tersebut, port 8270 berstatus **open**, sehingga service dapat dijangkau dari Kali Linux.

Pesan `inverse host lookup failed` tidak menjadi masalah karena koneksi ke port tetap berhasil.



Gambar 2. Hasil Pengecekan Koneksi Port Service

4.3 Identifikasi Traffic Scanning

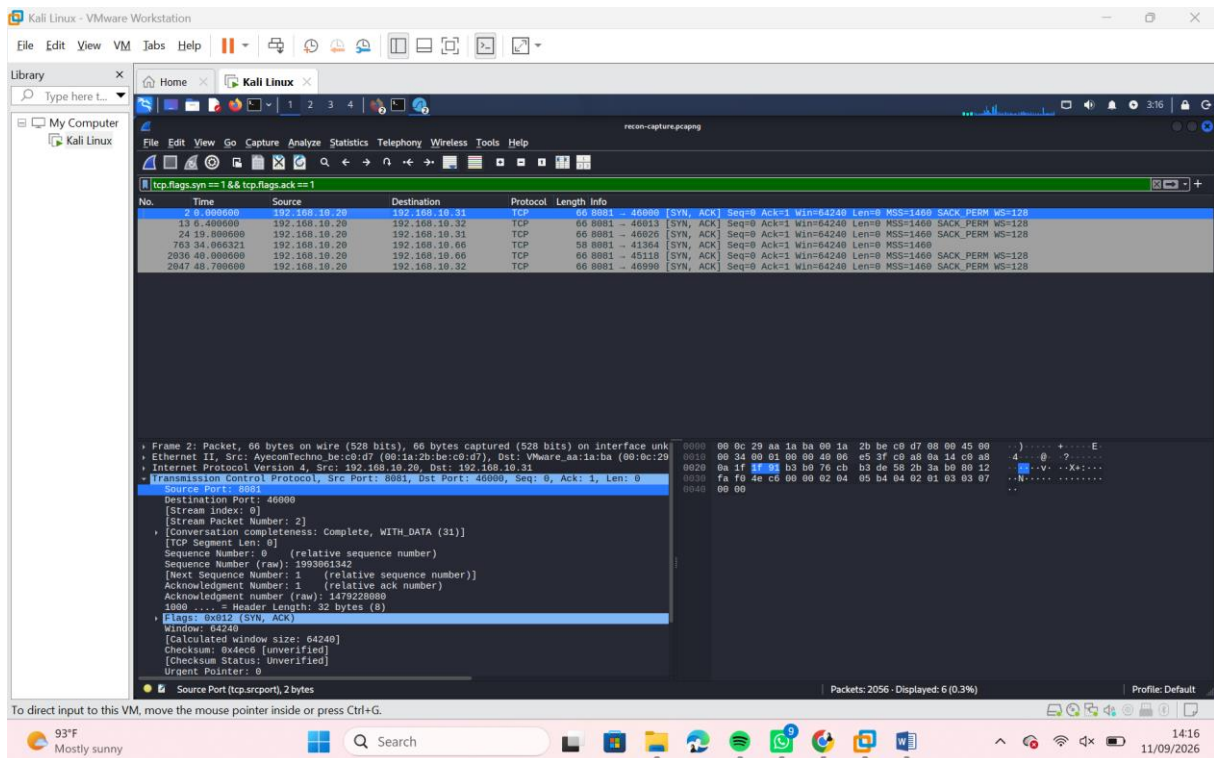
Capture dibuka menggunakan Wireshark. Untuk mencari paket yang merupakan percobaan koneksi TCP awal, digunakan filter:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

Filter tersebut digunakan untuk menampilkan paket **SYN** yang dikirim tanpa ACK, yang menunjukkan adanya percobaan koneksi awal ke suatu port.

Dari hasil analisis terlihat banyak paket SYN dari **192.168.10.66** menuju **192.168.10.20** dengan destination port yang berbeda-beda. Pola tersebut menunjukkan adanya aktivitas scanning terhadap beberapa port pada host target.

Hasil menunjukkan adanya banyak paket SYN dari **192.168.10.66** menuju **192.168.10.20** dengan destination port yang berbeda-beda. Hal tersebut menunjukkan adanya aktivitas scanning terhadap beberapa port pada host target.



Gambar 3. Hasil Identifikasi Traffic Scanning

4.4 Identifikasi IP attacker dan target

Berdasarkan traffic yang diamati, IP 192.168.10.66 mengirimkan banyak paket SYN menuju host 192.168.10.20.

Sehingga:

Parameter	Hasil
IP attacker	192.168.10.66
IP host yang di-scan	192.168.10.20

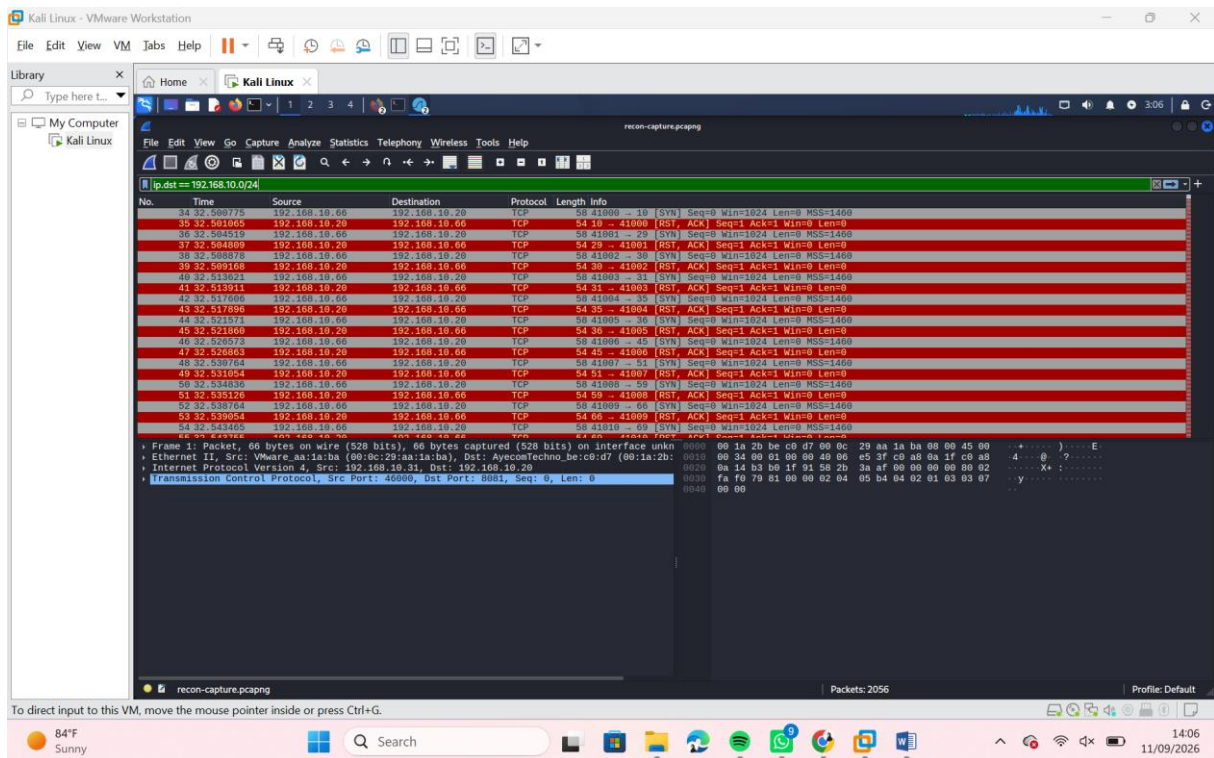
4.5 Identifikasi teknik scanning

Pola traffic menunjukkan attacker mengirimkan paket **TCP SYN** ke banyak destination port dan melihat response dari host target.

Pola tersebut merupakan:

TCP SYN Scan / SYN Scanning

Teknik ini digunakan untuk mengetahui status port tanpa melakukan koneksi TCP secara penuh.



Gambar 4. Hasil Identifikasi Teknik Scanning

4.6 Menghitung jumlah port unik yang diprobe

Command yang digunakan:

```
tshark -r ~/recon-capture.pcapng \
-Y "ip.src == 192.168.10.66 && tcp.flags.syn == 1 && tcp.flags.ack == 0" \
-T fields -e tcp.dstport | sort -n | uniq | wc -l
```

Command tersebut digunakan untuk mengambil seluruh destination port yang diprobe oleh attacker, menghapus port yang duplikat, kemudian menghitung jumlah port unik.

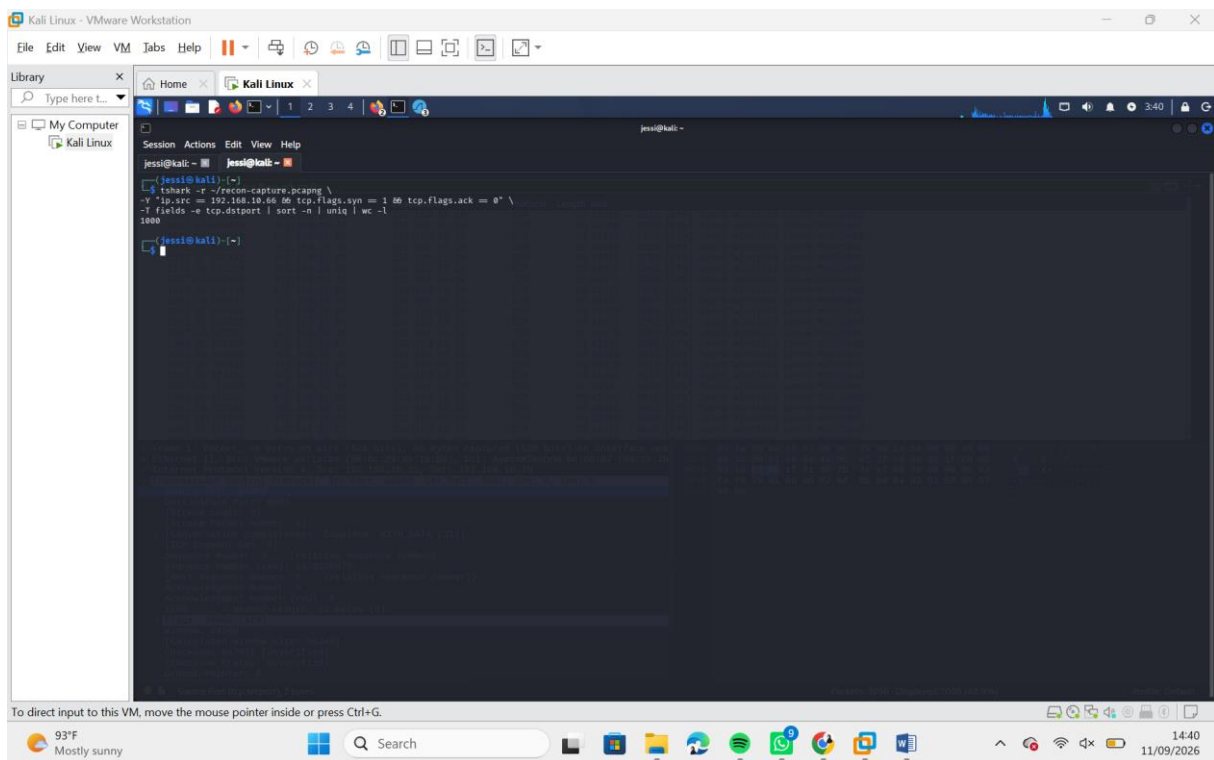
Penjelasan command:

- `-r` → membaca file capture.
- `-Y` → menggunakan display filter.
- `ip.src == 192.168.10.66` → hanya mengambil traffic dari attacker.
- `tcp.flags.syn == 1 && tcp.flags.ack == 0` → hanya mengambil paket SYN awal.
- `-T fields` → menampilkan field tertentu.
- `-e tcp.dstport` → mengambil destination port.
- `sort -n` → mengurutkan nomor port.
- `uniq` → menghapus duplikat yang berurutan.
- `wc -l` → menghitung jumlah port yang tersisa.

Hasil command menunjukkan terdapat:

1000 port unik

yang diprobe oleh attacker.



Gambar 5. Hasil menghitung jumlah Port unik yang diprobe

5. Menjalankan Service Challenge

Setelah melakukan analisis terhadap PCAP, service challenge dijalankan menggunakan:

```
nc 43.134.2.156 8270
```

Kemudian service menampilkan:

```
=====
Siapa yang Ngetok Pintu
=====
Insiden : lonjakan koneksi gagal di segmen 192.168.10.0/24
Artifact: recon-capture.pcapng
```

Jawab 5 pertanyaan berikut berdasarkan isi capture.

Service kemudian memberikan lima pertanyaan yang harus dijawab berdasarkan hasil analisis PCAP.

```
(jessi@kali)-[~]
$ nc 43.134.2.156 8270

Siapa yang Ngetok Pintu

Insiden : lonjakan koneksi gagal di segmen 192.168.10.0/24
Artifact: recon-capture.pcapng

Jawab 5 pertanyaan berikut berdasarkan isi capture.

Q1) IP address penyerang (yang melakukan scanning)? > 192.168.10.66
[+] Benar.

Q2) IP address host yang di-scan? > 192.168.10.20
[+] Benar.

Q3) Teknik scanning yang dipakai (nama teknik, bukan nama tools)? > SYN
[+] Benar.

Q4) Nomor port yang statusnya OPEN di host target? > 8081
[+] Benar.

Q5) Ada berapa port unik yang di-probe oleh penyerang? > 1000
[+] Benar.

Semua jawaban benar.
Flag: FLAG{syn_sc4n_f00tpr1nt_1d3nt1f13d}
```

Gambar 6. Hasil menjalankan Service Challenge

6. Hasil Akhir

Setelah seluruh pertanyaan dijawab dengan benar, service memberikan hasil:

```
=====
Semua jawaban benar.
Flag: FLAG{syn_sc4n_f00tpr1nt_1d3nt1f13d}
=====
```

Flag yang diperoleh:

FLAG{syn_sc4n_f00tpr1nt_1d3nt1f13d}

7. Conclusion

Berdasarkan hasil analisis network traffic, IP **192.168.10.66** teridentifikasi sebagai attacker yang melakukan scanning terhadap **192.168.10.20**. Aktivitas tersebut menggunakan teknik **TCP SYN Scan**, yang terlihat dari banyaknya paket SYN menuju berbagai destination port. Perhitungan menggunakan TShark menunjukkan terdapat **1000 port unik** yang diprobe oleh attacker.

Weird Domain Name

1. Objective

Melakukan analisis network traffic pada file `dns.traffic.pcapng` untuk mengidentifikasi aktivitas DNS yang mencurigakan pada jaringan finance. Analisis difokuskan pada pola query DNS, domain yang dituju, workstation yang melakukan komunikasi, serta isi subdomain untuk mengetahui apakah terdapat data yang dikirim melalui traffic DNS.

2. Target

- **Artifact:** `dns.traffic.pcapng`
- **Target:** Traffic DNS pada jaringan finance

3. Tools yang Digunakan

- **Kali Linux**
- **TShark** untuk menganalisis traffic DNS dan mengambil field dari packet capture
- **Base32 decoder** untuk melakukan decoding terhadap payload yang ditemukan

4. Langkah Pengerjaan

4.1 Membuka dan Menganalisis Traffic DNS

File `dns.traffic.pcapng` digunakan sebagai artifact untuk melakukan analisis network traffic.

Analisis awal dilakukan menggunakan TShark untuk melihat nama domain yang muncul pada query DNS.

Command yang digunakan:

```
tshark -r dns.traffic.pcapng -Y "dns.qry.name" -T fields -e dns.qry.name |  
sort | uniq -c | sort -nr
```

Command tersebut digunakan untuk mengambil nama domain dari DNS query, kemudian mengelompokkan dan menghitung jumlah kemunculannya.

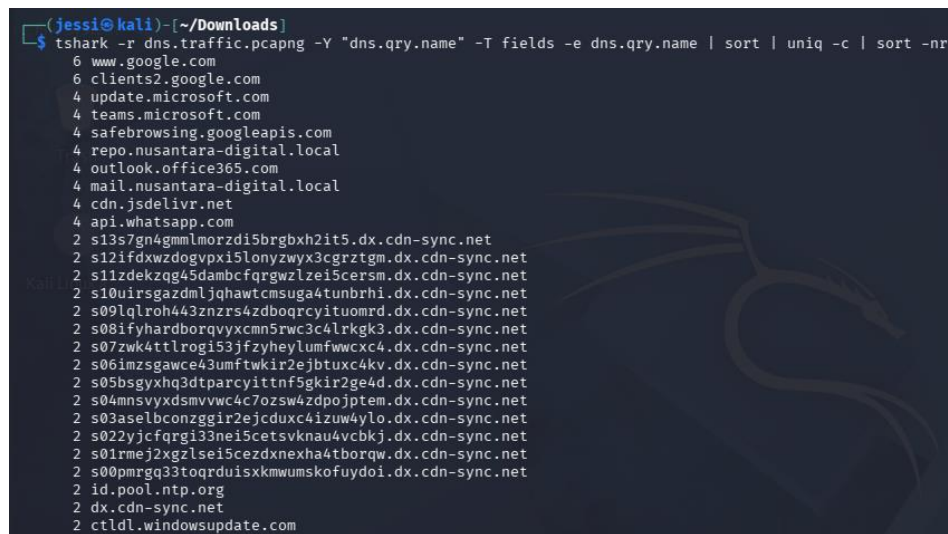
Dari hasil analisis ditemukan beberapa domain yang umum digunakan, seperti:

- `www.google.com`
- `update.microsoft.com`
- `teams.microsoft.com`
- `outlook.office365.com`

Namun, ditemukan juga domain yang berbeda dari domain normal lainnya, yaitu:

`dx.cdn-sync.net`

Domain tersebut memiliki beberapa subdomain dengan pola yang tidak biasa sehingga dilakukan analisis lebih lanjut.



```
(jessi@kali)~[~/Downloads]
tshark -r dns.traffic.pcapng -Y "dns.qry.name" -T fields -e dns.qry.name | sort | uniq -c | sort -nr
6 www.google.com
6 clients2.google.com
4 update.microsoft.com
4 teams.microsoft.com
4 safebrowsing.googleapis.com
4 repo.nusantara-digital.local
4 outlook.office365.com
4 mail.nusantara-digital.local
4 cdn.jsdelivr.net
4 api.whatsapp.com
2 s13s7gn4gmmlmorzdi5brgbxh2it5.dx.cdn-sync.net
2 s12ifdxwzdogvpxi5lonyzwyx3cgrztgm.dx.cdn-sync.net
2 s11zdekzqg45dambcfqrgwzlzei5cersm.dx.cdn-sync.net
2 s10uirsgazdmljqhawtcsuga4tunbrhi.dx.cdn-sync.net
2 s09lqlroh443znzrs4zdboqrcytiomrd.dx.cdn-sync.net
2 s08ifyhardborqvvyxcmn5rwc3c4lrkgk3.dx.cdn-sync.net
2 s07zwwk4ttlrogi53jfzyheylumfwwxc4.dx.cdn-sync.net
2 s06imzsgawce43umftwki2ejbtuxc4kv.dx.cdn-sync.net
2 s05bsgyxhq3dtparcyitn5gk2ge4d.dx.cdn-sync.net
2 s04mnsvyxdsmlvvc4c7ozsw4zdpjptem.dx.cdn-sync.net
2 s03asclbconzggr2ejcdx4izuw4ylo.dx.cdn-sync.net
2 s022yjcfrgi33nei5cetsvknau4vcbkj.dx.cdn-sync.net
2 s01rmej2xgzlsei5cezdxe4tborqw.dx.cdn-sync.net
2 s00pmrgq33toqrduisxkmwumskofuydoi.dx.cdn-sync.net
2 id.pool.ntp.org
2 dx.cdn-sync.net
2 ctldl.windowsupdate.com
```

Gambar 1. Hasil Identifikasi Domain DNS

4.2 Identifikasi Workstation

Selanjutnya dilakukan analisis untuk mengetahui workstation yang melakukan query terhadap domain `cdn-sync.net`.

Command yang digunakan:

```
tshark -r dns.traffic.pcapng -Y "dns.qry.name contains cdn-sync.net" -T fields -e frame.number -e frame.time -e ip.src -e dns.qry.name
```

Hasil analisis menunjukkan bahwa query terhadap domain tersebut berasal dari:

10.10.20.53

Dari traffic juga terlihat adanya query secara berurutan terhadap subdomain seperti:

```
s00pmrgq33toqrduisxkmwumskofuydoi.dx.cdn-sync.net
s01rmej2xgzlsei5cezdxe4tborqw.dx.cdn-sync.net
```

hingga:

```
s13s7gn4gmmlmorzdi5brgbxh2it5.dx.cdn-sync.net
```

Adanya pola sequence `s00` sampai `s13` menunjukkan bahwa bagian setelah nomor sequence kemungkinan digunakan sebagai potongan data yang dikirim secara berurutan melalui DNS query.

```
(jessi@kali) - [~/Downloads]
$ tshark -r dns.traffic.pcapng -Y "dns.qry.name contains cdn-sync.net" -T fields -e frame.number -e frame.time -e ip.src -e dns.qry.name
29 2026-08-06T05:07:22.881621000-0400 10.10.20.53 dx.cdn-sync.net
30 2026-08-06T05:07:22.898020000-0400 10.10.20.1 dx.cdn-sync.net
31 2026-08-06T05:07:23.781621000-0400 10.10.20.53 s00pmrgq33toqrduisxkmwmskofuydoi.dx.cdn-sync.net
32 2026-08-06T05:07:23.847819000-0400 10.10.20.1 s00pmrgq33toqrduisxkmwmskofuydoi.dx.cdn-sync.net
33 2026-08-06T05:07:25.580229000-0400 10.10.20.53 s01rmej2xgzlsei5cezdxdnexus4tborqw.dx.cdn-sync.net
34 2026-08-06T05:07:25.658821000-0400 10.10.20.1 s01rmej2xgzlsei5cezdxdnexus4tborqw.dx.cdn-sync.net
35 2026-08-06T05:07:27.643196000-0400 10.10.20.53 s022yjcfrgi33nei5cetsvknau4vcbkj.dx.cdn-sync.net
36 2026-08-06T05:07:27.700892000-0400 10.10.20.1 s022yjcfrgi33nei5cetsvknau4vcbkj.dx.cdn-sync.net
37 2026-08-06T05:07:29.907124000-0400 10.10.20.53 s03aselbconzggir2ejcduxc4izuw4ylo.dx.cdn-sync.net
38 2026-08-06T05:07:29.965386000-0400 10.10.20.1 s03aselbconzggir2ejcduxc4izuw4ylo.dx.cdn-sync.net
39 2026-08-06T05:07:32.117559000-0400 10.10.20.53 s04mnsyvxdsmyvwc4c7ozsw4zdpjptem.dx.cdn-sync.net
40 2026-08-06T05:07:32.165476000-0400 10.10.20.1 s04mnsyvxdsmyvwc4c7ozsw4zdpjptem.dx.cdn-sync.net
41 2026-08-06T05:07:33.972503000-0400 10.10.20.53 s05bsgyxhq3dtparcyittnf5gkir2ge4d.dx.cdn-sync.net
42 2026-08-06T05:07:34.007860000-0400 10.10.20.1 s05bsgyxhq3dtparcyittnf5gkir2ge4d.dx.cdn-sync.net
43 2026-08-06T05:07:36.222470000-0400 10.10.20.53 s06imzsgawce43umftwkir2ejbtuxc4kv.dx.cdn-sync.net
44 2026-08-06T05:07:36.274680000-0400 10.10.20.1 s06imzsgawce43umftwkir2ejbtuxc4kv.dx.cdn-sync.net
45 2026-08-06T05:07:38.334686000-0400 10.10.20.53 s07zkw4ttlrogi53jfzyheylumfwxc4c4.dx.cdn-sync.net
46 2026-08-06T05:07:38.389694000-0400 10.10.20.1 s07zkw4ttlrogi53jfzyheylumfwxc4c4.dx.cdn-sync.net
47 2026-08-06T05:07:40.234569000-0400 10.10.20.53 s08ifyhardborqvyxcmn5rwc3c4lrkgk3.dx.cdn-sync.net
48 2026-08-06T05:07:40.301258000-0400 10.10.20.1 s08ifyhardborqvyxcmn5rwc3c4lrkgk3.dx.cdn-sync.net
49 2026-08-06T05:07:42.197113000-0400 10.10.20.53 s09lqlroh443znzrs4zdboqrcyituomrd.dx.cdn-sync.net
50 2026-08-06T05:07:42.247829000-0400 10.10.20.1 s09lqlroh443znzrs4zdboqrcyituomrd.dx.cdn-sync.net
51 2026-08-06T05:07:44.492747000-0400 10.10.20.53 s10uirsgazdmljqhawtcmuga4tunbrhi.dx.cdn-sync.net
52 2026-08-06T05:07:44.550017000-0400 10.10.20.1 s10uirsgazdmljqhawtcmuga4tunbrhi.dx.cdn-sync.net
53 2026-08-06T05:07:46.660433000-0400 10.10.20.53 s11zdekzqg45dambcfqrgwzlei5cersm.dx.cdn-sync.net
54 2026-08-06T05:07:46.746481000-0400 10.10.20.1 s11zdekzqg45dambcfqrgwzlei5cersm.dx.cdn-sync.net
55 2026-08-06T05:07:48.506145000-0400 10.10.20.53 s12ifdxwzdogvpxi5lonzywx3cgrztgm.dx.cdn-sync.net
56 2026-08-06T05:07:48.587171000-0400 10.10.20.1 s12ifdxwzdogvpxi5lonzywx3cgrztgm.dx.cdn-sync.net
57 2026-08-06T05:07:50.431451000-0400 10.10.20.53 s13s7gn4gmmlmorzd15brgbxh2it5.dx.cdn-sync.net
58 2026-08-06T05:07:50.475589000-0400 10.10.20.1 s13s7gn4gmmlmorzd15brgbxh2it5.dx.cdn-sync.net
```

Gambar 2. Hasil Identifikasi Workstation dan DNS Query

4.3 Mengambil Subdomain yang Mencurigakan

Untuk mengambil subdomain yang memiliki pola sequence tersebut, digunakan command:

```
tshark -r dns.traffic.pcapng -Y "dns.qry.name contains cdn-sync.net" -T fields -e dns.qry.name | grep '^s[0-9]' | sort -u
```

Hasil command menunjukkan adanya **14 bagian data**, mulai dari s00 hingga s13.

Setiap bagian memiliki karakter yang terlihat acak dan menggunakan karakter yang sesuai dengan pola Base32. Oleh karena itu, bagian payload perlu digabungkan berdasarkan urutan sequence sebelum dilakukan decoding.

```
(jessi@kali) - [~/Downloads]
$ tshark -r dns.traffic.pcapng -Y "dns.qry.name contains cdn-sync.net" -T fields -e dns.qry.name | grep '^s[0-9]' | sort -u
s00pmrgq33toqrduisxkmwmskofuydoi.dx.cdn-sync.net
s01rmej2xgzlsei5cezdxdnexus4tborqw.dx.cdn-sync.net
s022yjcfrgi33nei5cetsvknau4vcbkj.dx.cdn-sync.net
s03aselbconzggir2ejcduxc4izuw4ylo.dx.cdn-sync.net
s04mnsyvxdsmyvwc4c7ozsw4zdpjptem.dx.cdn-sync.net
s05bsgyxhq3dtparcyittnf5gkir2ge4d.dx.cdn-sync.net
s06imzsgawce43umftwkir2ejbtuxc4kv.dx.cdn-sync.net
s07zkw4ttlrogi53jfzyheylumfwxc4c4.dx.cdn-sync.net
s08ifyhardborqvyxcmn5rwc3c4lrkgk3.dx.cdn-sync.net
s09lqlroh443znzrs4zdboqrcyituomrd.dx.cdn-sync.net
s10uirsgazdmljqhawtcmuga4tunbrhi.dx.cdn-sync.net
s11zdekzqg45dambcfqrgwzlei5cersm.dx.cdn-sync.net
s12ifdxwzdogvpxi5lonzywx3cgrztgm.dx.cdn-sync.net
s13s7gn4gmmlmorzd15brgbxh2it5.dx.cdn-sync.net
```

Gambar 3. Hasil Pengambilan Subdomain s00–s13

4.4 Menggabungkan dan Decode Payload

Setelah seluruh bagian ditemukan, payload digabungkan berdasarkan urutan s00 sampai s13.

Command yang digunakan:

```
tshark -r dns.traffic.pcapng -Y "dns.qry.name contains cdn-sync.net" -T fields -e dns.qry.name | grep '^s[0-9]' | sort -u | sort -k1.2,1.3n | sed 's/^s[0-9][0-9]//' | sed 's/\.dx\.cdn-sync\.net$//' | tr -d '\n' | tr '[:lower:]' '[:upper:]' | base32 -d
```

Command tersebut digunakan untuk:

- mengambil DNS query yang berkaitan dengan `cdn-sync.net`;
- mengambil subdomain dengan sequence `s00` sampai `s13`;
- mengurutkan bagian berdasarkan nomor sequence;
- menghapus nomor sequence;
- menghapus nama domain `dx.cdn-sync.net`;
- menggabungkan seluruh payload;
- mengubah karakter menjadi uppercase;
- melakukan decoding menggunakan Base32.

Hasil decoding:

```
{ "host": "WS-FIN-07", "user": "dwi.pratama", "dom": "NUSANTARA", "src": "D:\\Finance\\rekap_vendor_2026.xlsx", "size": 184320, "stage": "C:\\Users\\dwi.pratama\\AppData\\Local\\Temp\\~sync.dat", "ts": "2026-08-12T09:41:22+07:00", "key": "FLAG{dn5_tunn3l_b4s32_3xf1ltr4t10n}" }
```

Dari hasil tersebut ditemukan beberapa informasi:

Parameter	Hasil
Workstation	WS-FIN-07
User	dwi.pratama
Domain	NUSANTARA
Source File	D:\Finance\rekap_vendor_2026.xlsx
File Size	184320 byte
Stage File	C:\Users\dwi.pratama\AppData\Local\Temp\~sync.dat
Timestamp	2026-08-12T09:41:22+07:00
Flag	FLAG{dn5_tunn3l_b4s32_3xf1ltr4t10n}

```

(jessi@kali)~/Downloads
$ tshark -r dns.traffic.pcapng -Y "dns.qry.name contains cdn-sync.net" -T fields -e dns.qry.name | grep 's[0-9]' | sort -u | sort -k1.2,1.3n | sed 's/^s[0-9][0-9]//' | sed 's/\.dx\.cdn-sync\.net$//' | tr -d '\n'
pmrgq32toqrduisxkmwmskofuydoirneJ2xglse15cerdzxnexha4tborqW2yjcFqgl33nei5cetsvknau4vcbkJasclbconzgglr2ejcduxc4izuw4ylomnsvyxdsmvvwc4c7ozsw4zdpjptembsgyxhq3dtparcyittnf5gklr2ge4dimzsgawce43umftwk1r2ejbtuxc4kvz
wKattlrog153jfrzyhylumfwex04ifyhardborqvyxcm5rwc3c4lrkgk3qlroh443znzrs4zdbqrcyituomrduirsgazdm1jqhawtcmSuga4tunbrhizdekzqg45dambcfqrqgw1ze15cersmifdxwzdogvpx15lonzywyx3cgrztgms7gn4gmmLmorzd15brgbxh2it5
(jessi@kali)~/Downloads
$ tshark -r dns.traffic.pcapng -Y "dns.qry.name contains cdn-sync.net" -T fields -e dns.qry.name | grep 's[0-9]' | sort -u | sort -k1.2,1.3n | sed 's/^s[0-9][0-9]//' | sed 's/\.dx\.cdn-sync\.net$//' | tr -d '\n' | tr '[:lower:]' '[:upper:]' | base32 -d
{"host":"WS-FIN-07","user":"dwi.pratama","dom":"NUSANTARA","src":"D:\\Finance\\rekap_vendor_2026.xlsx","size":184320,"stage":"C:\\Users\\dwi.pratama\\AppData\\Local\\Temp\\-sync.dat","ts":"2026-08-12T09:41:22+07:00","key":"FLAG{dn5_tunn3l_b4s32_3xf1ltr4t10n}" }
(jessi@kali)~/Downloads
$

```

Gambar 4. Hasil Decoding Payload

5. Hasil Analisis

Berdasarkan hasil analisis traffic DNS, ditemukan komunikasi mencurigakan dari workstation:

10.10.20.53

yang kemudian teridentifikasi sebagai:

WS-FIN-07

Workstation tersebut melakukan query secara berurutan menuju:

dx.cdn-sync.net

dengan subdomain s00 sampai s13.

Pola tersebut menunjukkan bahwa subdomain digunakan untuk membawa potongan data. Setelah seluruh bagian digabungkan dan didecode menggunakan Base32, ditemukan informasi mengenai workstation, user, file sumber, serta flag.

Data hasil decoding juga menunjukkan adanya file:

D:\Finance\rekap_vendor_2026.xlsx

dengan ukuran:

184320 byte

Pola pengiriman data melalui beberapa subdomain secara berurutan dan penggunaan encoding Base32 menunjukkan indikasi penggunaan **DNS tunneling** sebagai media komunikasi dan kemungkinan **data exfiltration**.

6. Flag

Flag yang berhasil ditemukan dari hasil decoding adalah:

FLAG{dn5_tunn3l_b4s32_3xf1ltr4t10n}

7. Conclusion

Berdasarkan hasil analisis dns.traffic.pcapng, ditemukan aktivitas DNS yang mencurigakan dari workstation **10.10.20.53** yang teridentifikasi sebagai **WS-FIN-07** menuju domain **dx.cdn-sync.net**.

Traffic tersebut menggunakan subdomain berurutan dari **s00 hingga s13** yang berisi potongan data dengan pola Base32. Setelah seluruh bagian digabungkan dan didecode, ditemukan informasi mengenai workstation, user, file sumber, serta flag.

Pola tersebut mengindikasikan penggunaan DNS sebagai media untuk membawa data secara tersembunyi, sehingga aktivitas ini dapat dikategorikan sebagai indikasi **DNS tunneling** dan kemungkinan **data exfiltration**.

Flag yang diperoleh adalah:

```
FLAG{dn5_tunn3l_b4s32_3xf1l1tr4t10n}
```

Breached

1. Objective

Menganalisis `access.log` untuk mengidentifikasi aktivitas mencurigakan pada website, mengetahui tahapan serangan yang dilakukan attacker, serta menemukan indikasi penggunaan **web shell** dan command execution melalui file PHP yang di-upload ke server.

2. Target

- **Artifact:** `access.log`
- **Target:** Web server perusahaan
- **Attacker IP:** `45.13.88.204`

3. Tools

- Kali Linux
- `grep` untuk mencari teks atau pola tertentu pada file atau output
- `base64 -d` untuk melakukan decoding data Base64
- Text editor / terminal

4. Langkah Pengerjaan

4.1 Mencari aktivitas pada direktori `/uploads`

Pertama, mencari request yang berkaitan dengan direktori `/uploads`:

```
grep -i "/uploads" access.log
```

Dari hasil log ditemukan aktivitas dari IP:

```
45.13.88.204
```

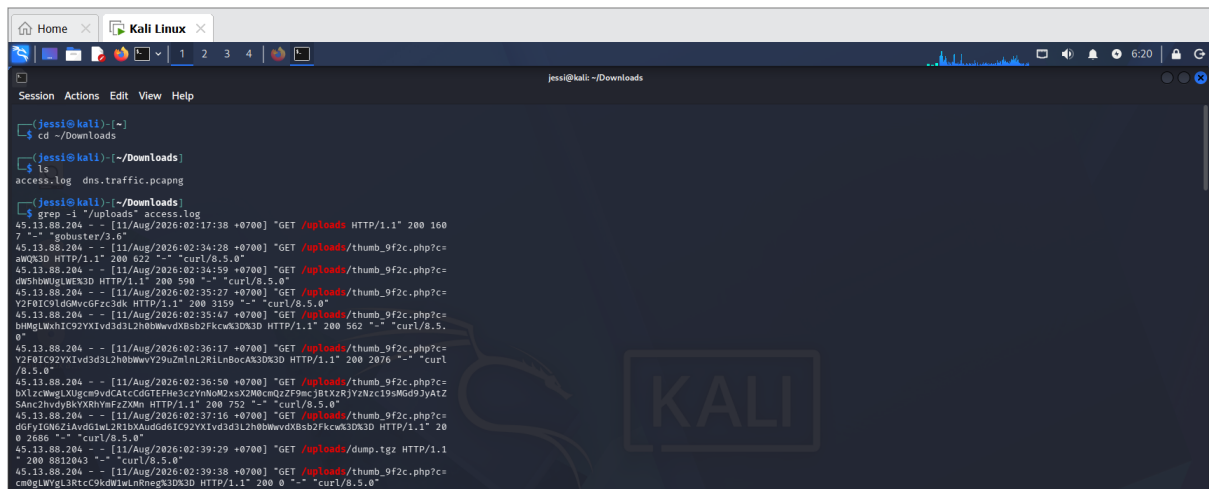
yang mengakses:

```
GET /uploads  
GET /admin/upload.php
```

dengan User-Agent:

```
gobuster/3.6
```

Penggunaan Gobuster untuk mengakses beberapa path menunjukkan adanya aktivitas **directory enumeration/reconnaissance** terhadap website.



```
jessi@kali:~$ cd ~/Downloads
jessi@kali:~/Downloads$ ls
access.log  dns.traffic.pcapng
jessi@kali:~/Downloads$ grep -i "/uploads" access.log
45.13.88.204 - - [11/Aug/2026:02:17:38 +0700] "GET /uploads HTTP/1.1" 200 160
7 "-" gobuster/3.0
45.13.88.204 - - [11/Aug/2026:02:34:28 +0700] "GET /uploads/thumb_9f2c.php?c=
awQk3D HTTP/1.1" 200 622 "-" "curl/8.5.0"
45.13.88.204 - - [11/Aug/2026:02:34:59 +0700] "GET /uploads/thumb_9f2c.php?c=
dmsHbmgLwE3D HTTP/1.1" 200 590 "-" "curl/8.5.0"
45.13.88.204 - - [11/Aug/2026:02:35:27 +0700] "GET /uploads/thumb_9f2c.php?c=
Y2F0IC9ldGwvcGFzc3dk HTTP/1.1" 200 3159 "-" "curl/8.5.0"
45.13.88.204 - - [11/Aug/2026:02:35:47 +0700] "GET /uploads/thumb_9f2c.php?c=
bmgLwLwIC92YXlvd3d3L2h0bWwvdXBsb2FkYXN3D HTTP/1.1" 200 562 "-" "curl/8.5.
0"
45.13.88.204 - - [11/Aug/2026:02:36:17 +0700] "GET /uploads/thumb_9f2c.php?c=
Y2F0IC92YXlvd3d3L2h0bWwvY29uZmLnL2RlLnBocAK3D HTTP/1.1" 200 2076 "-" "curl
/8.5.0"
45.13.88.204 - - [11/Aug/2026:02:36:50 +0700] "GET /uploads/thumb_9f2c.php?c=
bXlzcWwLdGwvcGFzc3dkICd0IFRheHJcZm9uZmLnL2RlLnBocAK3D HTTP/1.1" 200 2076 "-" "curl/8.5.0"
45.13.88.204 - - [11/Aug/2026:02:37:16 +0700] "GET /uploads/thumb_9f2c.php?c=
d0Y1Q0ZlAvdGwvcGFzc3dkICd0IFRheHJcZm9uZmLnL2RlLnBocAK3D HTTP/1.1" 200
0 2686 "-" "curl/8.5.0"
45.13.88.204 - - [11/Aug/2026:02:39:29 +0700] "GET /uploads/dump.tgz HTTP/1.1
200 8812043 "-" "curl/8.5.0"
45.13.88.204 - - [11/Aug/2026:02:39:38 +0700] "GET /uploads/thumb_9f2c.php?c=
cm0gWwLwLwIC92YXlvd3d3L2h0bWwvdXBsb2FkYXN3D HTTP/1.1" 200 0 "-" "curl/8.5.0"
```

Gambar 1. Hasil Aktivitas pada Direktori /uploads

4.2 Menelusuri aktivitas attacker

Selanjutnya, seluruh aktivitas dari IP attacker ditampilkan dengan:

```
grep "45.13.88.204" access.log
```

Dari hasil analisis terlihat bahwa attacker sebelumnya melakukan **SQL Injection** terhadap:

```
/produk.php?id=...
```

dengan menggunakan:

```
sqlmap
```

Attacker juga melakukan percobaan untuk mengambil informasi dari tabel `users`, termasuk `username` dan `password`.

Setelah itu, sekitar pukul **02:32–02:33**, attacker mengakses:

```
/admin/login.php
```

dan melakukan:

```
POST /admin/login.php
```

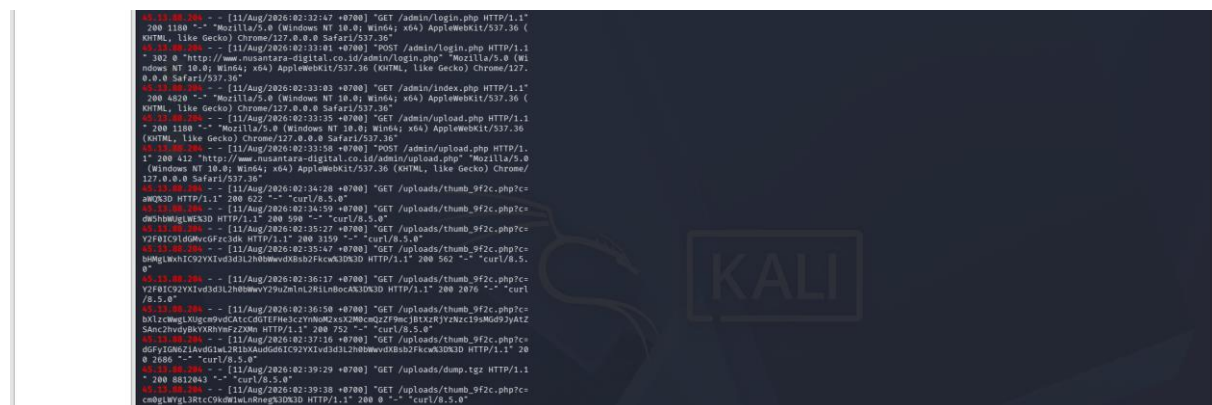
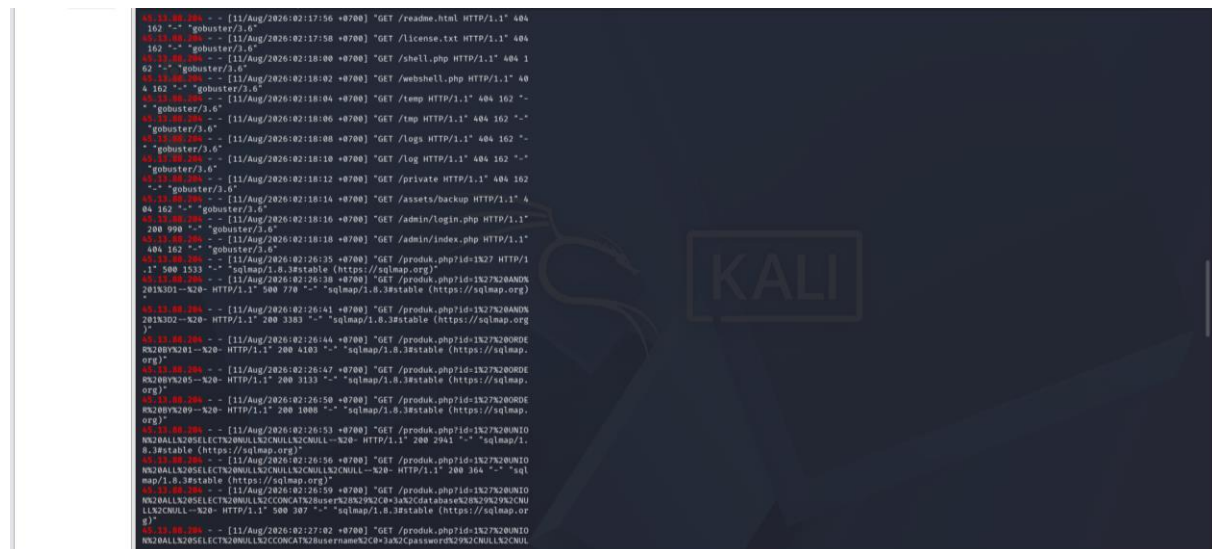
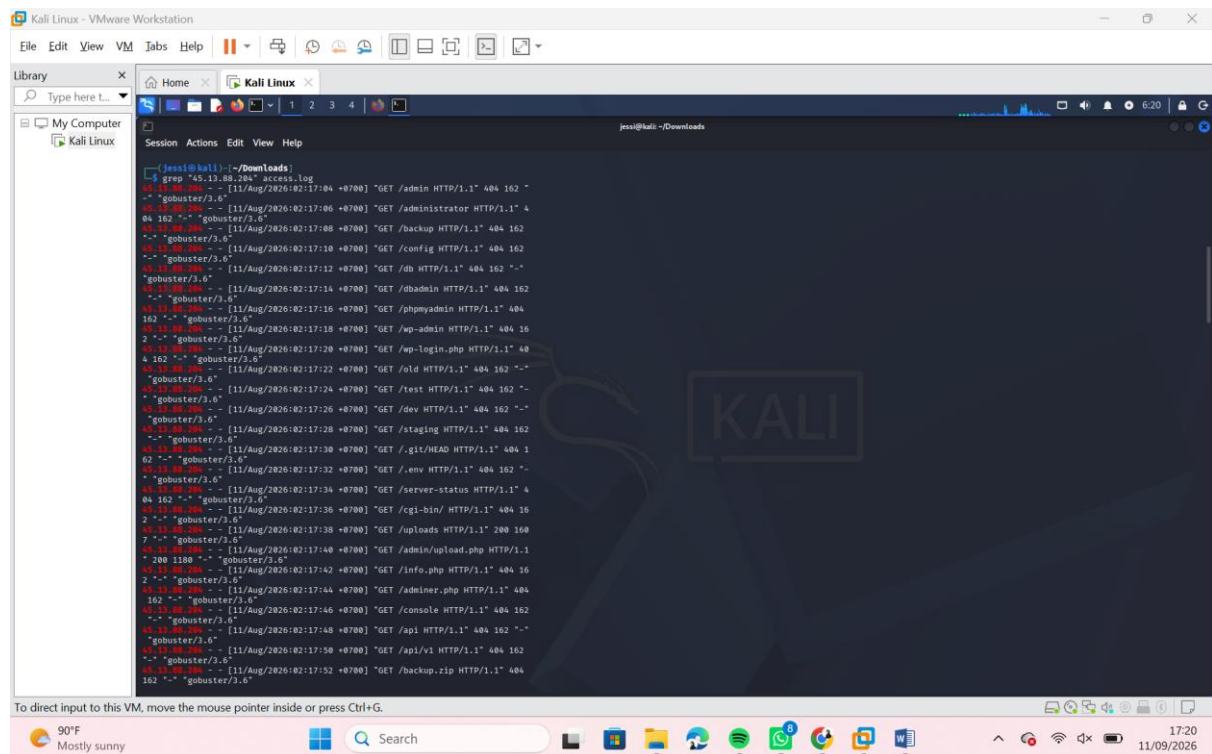
Request tersebut mendapatkan response:

```
302
```

Setelah itu, attacker mengakses:

```
GET /admin/index.php
```

Sehingga terdapat indikasi kuat bahwa attacker berhasil mendapatkan akses ke area admin.



Gambar 2. Hasil Menelusuri Aktivitas Attacker

5. Menemukan Flag

Salah satu request penting yang ditemukan pada access log adalah:

```
GET
/uploads/thumb_9f2c.php?c=bXlzcWwgLXUgcm9vdCAtcCdGTEFHe3czYnNoM2xsX2M0cmQzZF9mcjBtXzRjYzNzc19sMGd9JyAtZSAnc2hvdYBkYXRhYmFzZXMn
```

Nilai parameter `c` kemudian didecode menggunakan:

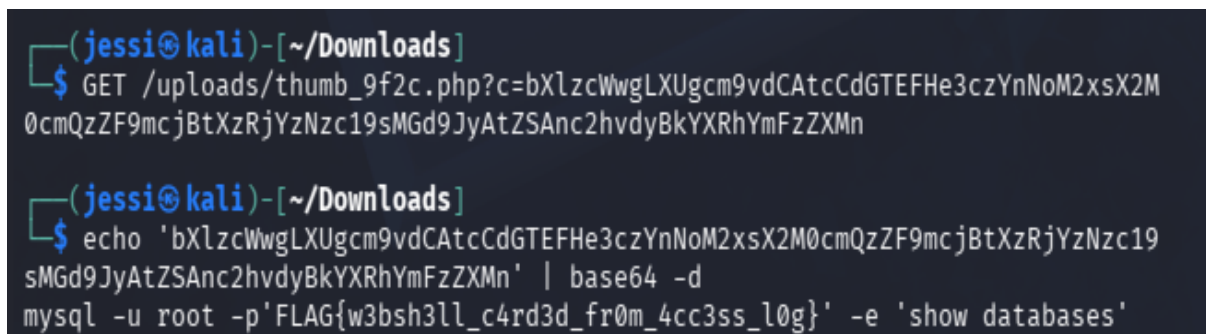
```
echo
'bXlzcWwgLXUgcm9vdCAtcCdGTEFHe3czYnNoM2xsX2M0cmQzZF9mcjBtXzRjYzNzc19sMGd9JyAtZSAnc2hvdYBkYXRhYmFzZXMn' | base64 -d
```

Hasil decoding:

```
mysql -u root -p'FLAG{w3bshell_c4rd3d_fr0m_4ccd3ss_10g}' -e 'show
databases'
```

Dari command tersebut ditemukan flag:

```
FLAG{w3bshell_c4rd3d_fr0m_4ccd3ss_10g}
```



```
(jessi@kali)-[~/Downloads]
$ GET /uploads/thumb_9f2c.php?c=bXlzcWwgLXUgcm9vdCAtcCdGTEFHe3czYnNoM2xsX2M0cmQzZF9mcjBtXzRjYzNzc19sMGd9JyAtZSAnc2hvdYBkYXRhYmFzZXMn

(jessi@kali)-[~/Downloads]
$ echo 'bXlzcWwgLXUgcm9vdCAtcCdGTEFHe3czYnNoM2xsX2M0cmQzZF9mcjBtXzRjYzNzc19sMGd9JyAtZSAnc2hvdYBkYXRhYmFzZXMn' | base64 -d
mysql -u root -p'FLAG{w3bshell_c4rd3d_fr0m_4ccd3ss_10g}' -e 'show databases'
```

Gambar 3. Hasil Flag

6. Hasil Analisis

Indikator	Hasil
Attacker IP	45.13.88.204
Web shell	thumb_9f2c.php
Lokasi web shell	/uploads/thumb_9f2c.php
Upload time	11/Aug/2026 02:33:58 +0700
Web shell execution	02:34:28 dan seterusnya
User-Agent	curl/8.5.0
Teknik awal	SQL Injection
Tool SQLi	sqlmap
Archive	/uploads/dump.tgz
Flag	FLAG{w3bshell_c4rd3d_fr0m_4ccd3ss_10g}

7. Flag

```
FLAG{w3bshell_c4rd3d_fr0m_4ccd3ss_10g}
```

8. Conclusion

Berdasarkan analisis `access.log`, attacker dengan IP `45.13.88.204` melakukan reconnaissance menggunakan Gobuster dan kemudian melakukan SQL Injection menggunakan `sqlmap`. Setelah mendapatkan akses ke area admin, attacker melakukan upload file PHP `thumb_9f2c.php` ke direktori `/uploads`.

File tersebut kemudian digunakan sebagai **web shell** dengan parameter Base64 untuk menjalankan command pada server. Attacker melakukan beberapa aktivitas seperti membaca `/etc/passwd`, mengakses konfigurasi database, serta membuat archive `dump.tgz` dari direktori uploads yang kemudian diakses melalui website.

Dari parameter command yang ditemukan pada access log berhasil diperoleh flag:

```
FLAG{w3bshell_c4rd3d_fr0m_4ccd3ss_l0g}
```

Tamu Tak Diundang

1. Objective

Pengujian ini dilakukan untuk mengetahui adanya aktivitas brute force pada layanan SSH yang terbuka ke internet. Analisis dilakukan dengan memeriksa file `auth.log` untuk mengetahui IP address penyerang, akun yang berhasil ditembus, jumlah percobaan login yang gagal, serta akun baru yang dibuat setelah penyerang berhasil mendapatkan akses.

2. Informasi Target

- Target: 43.134.2.156:15531
- Layanan: SSH
- Artifact: `auth.log`
- Sumber log: `/var/log/auth.log`

3. Langkah Pengerjaan

3.1 Spawn Instance

Pertama, membuka halaman challenge:

<http://43.134.2.156:8000/instance/netsec-05-ssh-bruteforce>

Kemudian melakukan **Spawn Instance**.

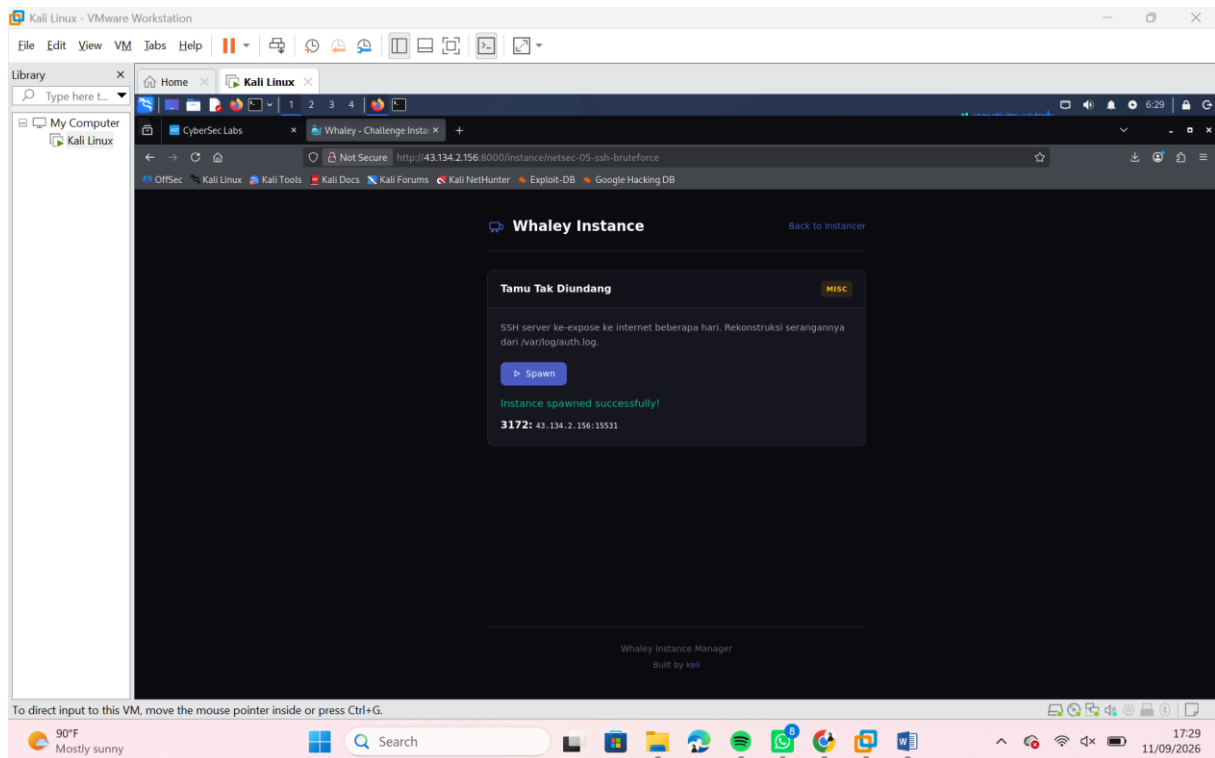
Setelah instance berhasil dibuat, muncul informasi:

```
Tamu Tak Diundang
SSH server ke-expose ke internet beberapa hari. Rekontruksi serangannya dari
/var/log/auth.log.
```

```
Instance spawned successfully!
3172 43.134.2.156:15531
```

Dari informasi tersebut diketahui bahwa service challenge berjalan pada:

```
IP: 43.134.2.156
Port: 15531
```



Gambar 1. Hasil Spawn Instance

3.2 Menentukan IP Address Penyerang

Untuk mencari aktivitas login SSH yang berhasil, digunakan perintah:

```
grep "Accepted password" auth.log
```

Dari hasilnya muncul:

```
Aug 12 03:13:24 srv-app01 sshd[13146]: Accepted password for backupsvc from 103.145.226.11 port 54118 ssh2
```

Perhatikan bagian:

```
from 103.145.226.11
```

Jadi IP penyerangnya adalah:

```
103.145.226.11
```

Di log tercatat bahwa login berhasil dengan:

```
Accepted password for backupsvc from 103.145.226.11
```

Jadi akun yang berhasil ditembus lewat brute force password adalah `backupsvc`.

```
(jessi@kali)-[~/Downloads]
$ grep "Accepted password" auth.log
Aug 12 03:13:24 srv-app01 sshd[13146]: Accepted password for backupsvc from 103.145.226.11 port 54118 ssh2
Aug 12 03:35:31 srv-app01 sshd[13183]: Accepted password for svcupdate from 103.145.226.11 port 49277 ssh2
```

Gambar 2. Hasil Menentukan IP Address Penyerang

3.3 Menghitung Percobaan Login yang Gagal

Setelah mendapatkan IP address penyerang, dilakukan pencarian terhadap seluruh percobaan login yang gagal dari IP tersebut menggunakan perintah:

```
grep "Failed password" auth.log | grep "103.145.226.11" | wc -l
```

Hasil perintah menunjukkan:

```
214
```

Artinya, terdapat 214 baris `Failed password` yang berasal dari IP 103.145.226.11 sebelum penyerang berhasil mendapatkan akses.

Penjelasannya:

- `grep "Failed password" auth.log` → mengambil semua percobaan login yang gagal.
- `grep "103.145.226.11"` → menyaring hanya dari IP penyerang.
- `wc -l` → menghitung jumlah baris.

```
(jessi@kali)-[~/Downloads]
$ grep "Failed password" auth.log | grep "103.145.226.11" | wc -l
214
```

Gambar 3. Hasil Menghitung Percobaan Login yang Gagal

3.4 Menentukan Akun Backdoor

Selanjutnya dilakukan pencarian terhadap aktivitas pembuatan akun baru menggunakan perintah:

```
grep "useradd" auth.log
```

Dari hasil log ditemukan perintah:

```
COMMAND=/usr/sbin/useradd -m -s /bin/bash -u 1337 svcupdate
```

Perhatikan bagian paling belakang:

```
svcupdate
```

Jadi username akun backdoor-nya adalah:

```
svcupdate
```

Perintah tersebut menunjukkan bahwa penyerang membuat akun baru dengan username `svcupdate`. Setelah itu, akun tersebut juga ditambahkan ke grup `sudo`, yang menunjukkan bahwa akun baru tersebut diberikan hak akses administratif.

Selain itu, pada log berikutnya juga ditemukan login menggunakan akun `svcupdate`, sehingga aktivitas tersebut semakin menunjukkan bahwa akun tersebut dibuat untuk digunakan kembali oleh penyerang.

```
(jessi@kali) - [~/Downloads]
$ grep "useradd" auth.log
Aug 12 03:17:05 srv-app01 sudo: backupsvc : TTY=pts/1 ; PWD=/home/backupsvc ; USER=root ; COMMAND=/usr/sbin/useradd -m -s /bin/bash -u 1337 svcupdate
Aug 12 03:17:05 srv-app01 useradd[13152]: new user: name=svcupdate, UID=1337, GID=1337, home=/home/svcupdate, shell=/bin/bash, from=/dev/pts/1
```

Gambar 4. Hasil Menentukan Akun Backdoor

4. Rekonstruksi Serangan

Berdasarkan urutan kejadian pada `auth.log`, aktivitas serangan dapat direkonstruksi sebagai berikut:

1. Penyerang dari IP `103.145.226.11` melakukan banyak percobaan login SSH menggunakan beberapa username.
2. Dari IP tersebut tercatat 214 percobaan `Failed password`.
3. Setelah beberapa kali percobaan, penyerang berhasil login menggunakan akun `backupsvc`.
4. Setelah berhasil masuk, akun `backupsvc` menjalankan beberapa perintah dengan hak akses `root`, termasuk membaca `/etc/shadow`.
5. Penyerang kemudian membuat akun baru bernama `svcupdate`.
6. Akun `svcupdate` ditambahkan ke grup `sudo` sehingga memiliki hak akses administratif.
7. Penyerang juga melakukan perubahan pada konfigurasi SSH dan melakukan restart terhadap service SSH.
8. Setelah itu, penyerang kembali melakukan login menggunakan akun `svcupdate`.

Dari rangkaian tersebut terlihat bahwa serangan tidak berhenti setelah password berhasil ditebak. Penyerang melanjutkan aksinya dengan membuat akun baru dan memberikan hak akses `sudo` sebagai bentuk persistence pada sistem.

5. Menjalankan Service Challenge

Setelah melakukan analisis terhadap PCAP, service challenge dijalankan menggunakan:

```
nc 43.134.2.156 15531
```

Kemudian service menampilkan:

```
=====
Tamuk Tak Diundang
=====
Insiden : SSH server ke-expose ke internet
```

Artifact: auth.log

Jawab 4 pertanyaan berikut berdasarkan isi capture.

Service kemudian memberikan empat pertanyaan yang harus dijawab berdasarkan hasil analisis PCAP.

```
(jessi@kali)~[/Downloads]
$ nc -v 43.134.2.156 15531
43.134.2.156: inverse host lookup failed: Unknown host
(UNKNOWN) [43.134.2.156] 15531 (?) open

=====
Tamu Tak Diundang
=====
Insiden : SSH server ke-expose ke internet
Artifact: auth.log

Jawab 4 pertanyaan berikut berdasarkan isi log.

Q1) IP address penyerang yang brute force-nya BERHASIL tembus? > 103.145.226.11
[+] Benar.

Q2) Akun mana yang berhasil ditembus lewat brute force password? > backupsvc
[+] Benar.

Q3) Berapa baris 'Failed password' yang berasal dari IP tersebut? > 214
[+] Benar.

Q4) Username akun backdoor yang dibuat penyerang setelah masuk? > svcupdate
[+] Benar.

=====
Semua jawaban benar.
Flag: FLAG{ssh_brut3f0rc3_th3n_p3rs1st3nc3}
```

Gambar 5. Hasil menjalankan Service Challenge

6. Kesimpulan

Berdasarkan hasil analisis auth.log, diketahui bahwa server SSH mengalami serangan brute force dari IP 103.145.226.11. Penyerang melakukan 214 percobaan login yang gagal hingga akhirnya berhasil mendapatkan akses menggunakan akun backupsvc.

Setelah berhasil masuk, penyerang melakukan aktivitas dengan hak akses root, membuat akun baru svcupdate, memberikan hak akses sudo, serta melakukan perubahan pada konfigurasi SSH. Hal ini menunjukkan bahwa setelah memperoleh akses awal, penyerang berusaha mempertahankan akses ke dalam sistem.

Berdasarkan rangkaian aktivitas tersebut, dapat disimpulkan bahwa terjadi indikasi **kompromi akun dan persistence** pada server SSH.

Last Packet Payroll

1. Objective

Menganalisis packet capture untuk mengidentifikasi aktivitas transfer file menggunakan protokol FTP, menemukan file yang ditransfer, mengidentifikasi TCP stream yang membawa file, serta mengekstrak dan memeriksa isi file untuk menemukan flag.

2. Target

- **Artifact:** ftp-session.pcapng
- **Workstation:** 192.168.30.44
- **FTP Server:** 192.168.30.9
- **Protocol:** FTP

3. Tools

- Kali Linux
- Wireshark untuk menganalisis network traffic dan file PCAP
- TShark untuk menganalisis traffic DNS dan mengambil field dari packet capture
- capinfos untuk melihat informasi dan metadata packet capture
- unzip untuk melihat isi dan mengekstrak file dari archive ZIP

4. Langkah Pengerjaan

4.1 Memeriksa File Packet Capture

Pertama, masuk ke folder tempat file PCAP disimpan:

```
cd ~/Downloads
```

Kemudian memastikan file tersedia:

```
ls -lh
```

File ftp-session.pcapng ditemukan pada folder Downloads.

Selanjutnya dilakukan pengecekan jenis file:

```
file ftp-session.pcapng
```

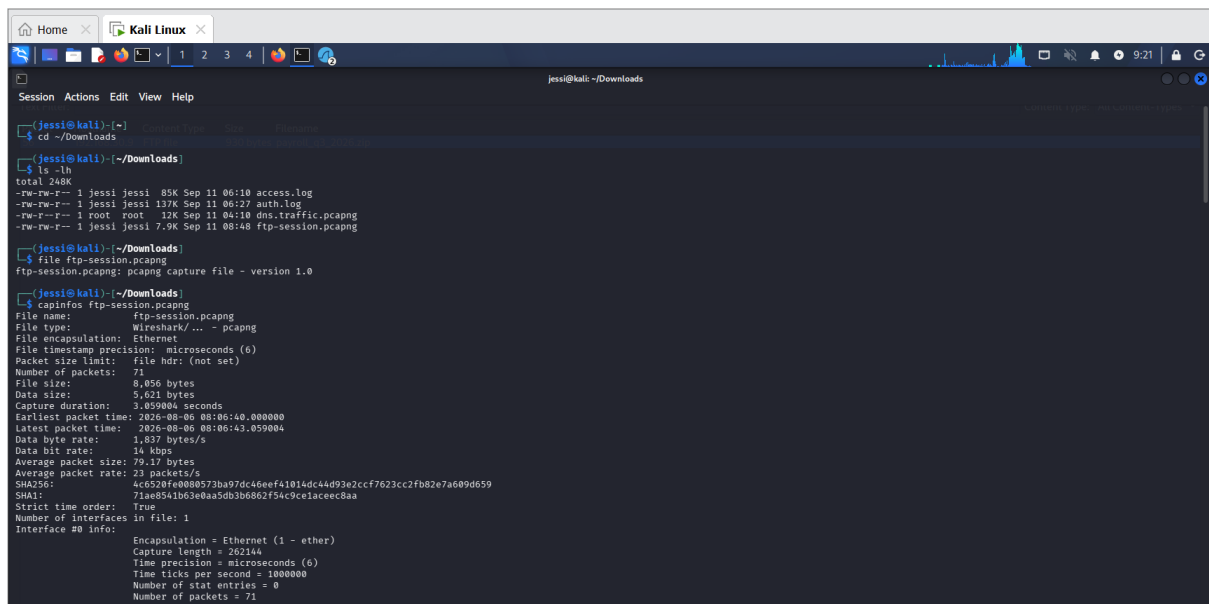
Hasil menunjukkan bahwa file merupakan:

```
pcapng capture file - version 1.0
```

Kemudian dilakukan pemeriksaan informasi packet capture menggunakan:

```
capinfos ftp-session.pcapng
```

Dari hasil tersebut diketahui bahwa packet capture memiliki **71 packet**, ukuran sekitar **8 KB**, dengan durasi sekitar **3 detik**.



Gambar 1. Hasil Memeriksa File Packet Capture

4.2 Mengidentifikasi Traffic FTP

Untuk mencari komunikasi yang menggunakan protokol FTP, digunakan:

```
tshark -r ftp-session.pcapng -Y "ftp"
```

Dari hasil analisis ditemukan komunikasi antara:

```
192.168.30.44 ↔ 192.168.30.9
```

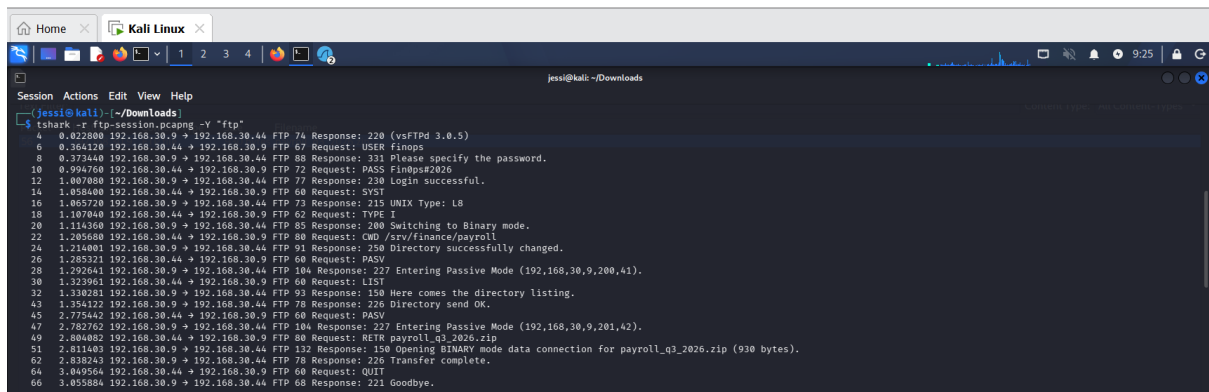
Diketahui bahwa:

- 192.168.30.44 merupakan workstation.
- 192.168.30.9 merupakan FTP server.
- Port 21 digunakan sebagai FTP control connection.
- FTP server menggunakan **vsFTPD 3.0.5**.

Pada traffic FTP ditemukan proses login:

```
USER finops
PASS Fin0ps#2026
230 Login successful.
```

Hal ini menunjukkan bahwa username dan password FTP dapat terlihat langsung pada packet capture karena komunikasi FTP tidak menggunakan enkripsi.



Gambar 2. Hasil Mengidentifikasi Traffic FTP

4.3 Mengidentifikasi File yang Ditransfer

Setelah login berhasil, client berpindah ke direktori:

```
/srv/finance/payroll
```

Hal tersebut terlihat dari command:

```
CWD /srv/finance/payroll
```

Kemudian client menjalankan:

```
LIST
```

untuk melihat daftar file.

Selanjutnya ditemukan command:

```
RETR payroll_q3_2026.zip
```

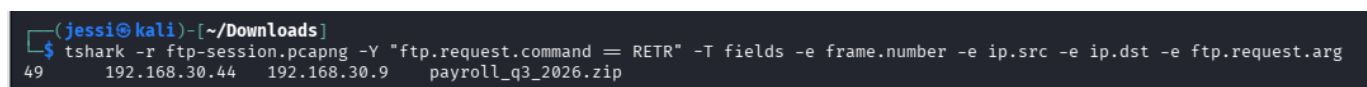
Command `RETR` menunjukkan bahwa client meminta file `payroll_q3_2026.zip` dari FTP server.

Untuk memastikan file yang diminta, digunakan:

```
tshark -r ftp-session.pcapng -Y "ftp.request.command == RETR" -T fields -e frame.number -e ip.src -e ip.dst -e ftp.request.arg
```

Hasil:

```
49      192.168.30.44      192.168.30.9      payroll_q3_2026.zip
```



Gambar 3. Hasil Mengidentifikasi File yang Ditransfer

Dari response FTP diketahui bahwa file payroll_q3_2026.zip memiliki ukuran 930 bytes:

```
(jessi@kali)-[~/Downloads]
$ tshark -r ftp-session.pcapng -Y "tcp.stream == 1" -T fields -e tcp.len | awk '{sum+=$1} END {print "Total bytes:",sum}'
Total bytes: 221

(jessi@kali)-[~/Downloads]
$ tshark -r ftp-session.pcapng -Y "tcp.stream == 2" -T fields -e tcp.len | awk '{sum+=$1} END {print "Total bytes:",sum}'
Total bytes: 930
```

Gambar 4. Hasil Mencari Ukuran File FTP

4.4 Mengidentifikasi TCP Stream

Selanjutnya dilakukan identifikasi TCP stream:

```
tshark -r ftp-session.pcapng -T fields -e tcp.stream | sort -n | uniq
```

Hasil:

```
0
1
2
```

```
(jessi@kali)-[~/Downloads]
$ tshark -r ftp-session.pcapng -T fields -e tcp.stream | sort -n | uniq
0
1
2
```

Gambar 5. Hasil Mengidentifikasi TCP Stream

Terdapat tiga TCP stream:

Stream	Fungsi
0	FTP control connection
1	FTP data connection untuk directory listing
2	FTP data connection untuk file payroll_q3_2026.zip

Selanjutnya, stream 1 dan stream 2 diperiksa untuk mengetahui koneksi yang digunakan pada masing-masing data stream:

```
tshark -r ftp-session.pcapng -Y "tcp.stream == 1" -T fields -e tcp.stream -
e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport
```

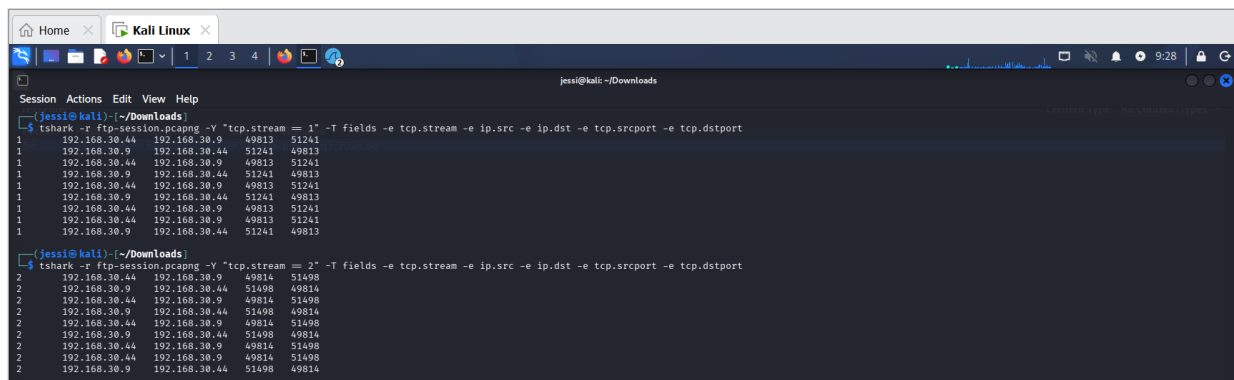
```
tshark -r ftp-session.pcapng -Y "tcp.stream == 2" -T fields -e tcp.stream -
e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport
```

Hasil menunjukkan koneksi:

```
192.168.30.44:49813
    ↑
192.168.30.9:51241

192.168.30.44:49814
    ↑
```

192.168.30.9:51498



Gambar 6. Hasil Memastikan Data pada 2 Stream

4.5 Memastikan Data pada Stream

Untuk memastikan bahwa stream 2 membawa file yang dimaksud, dilakukan perhitungan jumlah data TCP:

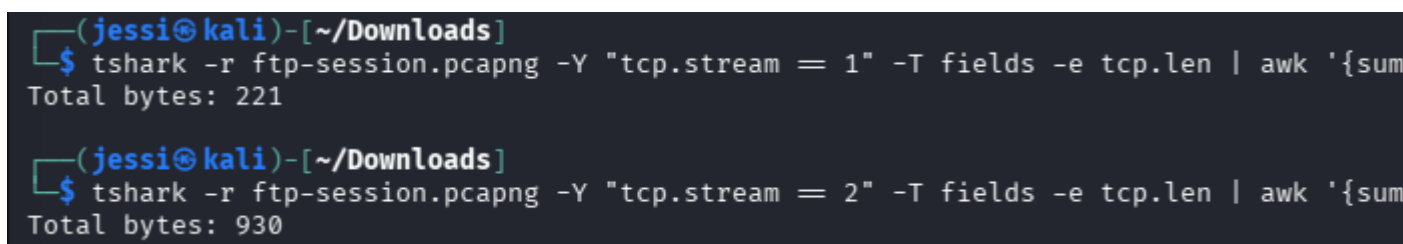
```
tshark -r ftp-session.pcapng -Y "tcp.stream == 2" -T fields -e tcp.len |  
awk '{sum+=$1} END {print "Total bytes:",sum}'
```

Hasil:

Total bytes: 930

Ukuran tersebut sesuai dengan ukuran payroll_q3_2026.zip yang tercatat pada response FTP.

Hal ini memperkuat bahwa **TCP stream 2 merupakan data transfer file payroll_q3_2026.zip.**



Gambar 7. Hasil Memastikan Data pada Stream

4.6 Export File dari Packet Capture

File kemudian diekstrak dari packet capture menggunakan Wireshark:

1. Membuka ftp-session.pcapng.
2. Memilih File → Export Objects → FTP Data.
3. Mencari payroll_q3_2026.zip.
4. Memilih file tersebut.
5. Menekan Save.

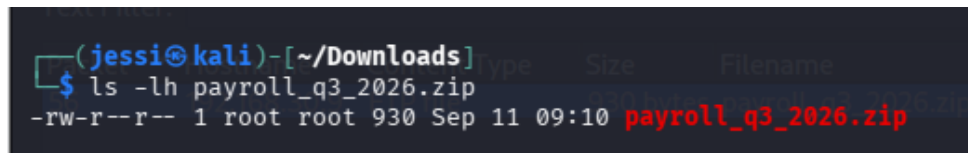
6. Menyimpan file ke folder Downloads.

Setelah proses export selesai, dilakukan pengecekan:

```
ls -lh payroll_q3_2026.zip
```

Hasil menunjukkan file berhasil diperoleh dengan ukuran:

```
930 bytes
```



```
(jessi@kali)-[~/Downloads]
$ ls -lh payroll_q3_2026.zip
-rw-r--r-- 1 root root 930 Sep 11 09:10 payroll_q3_2026.zip
```

Gambar 8. Hasil Export File dari Packet Capture

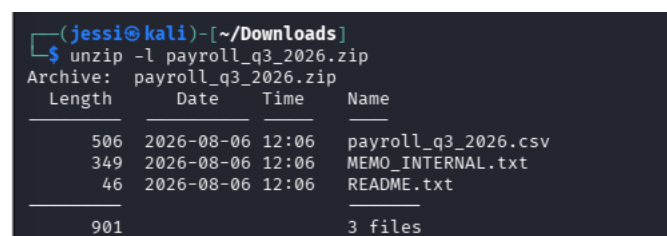
4.7 Memeriksa dan Mengekstrak Isi ZIP

Isi archive diperiksa menggunakan:

```
unzip -l payroll_q3_2026.zip
```

Ditemukan tiga file:

```
payroll_q3_2026.csv
MEMO_INTERNAL.txt
README.txt
```



```
(jessi@kali)-[~/Downloads]
$ unzip -l payroll_q3_2026.zip
Archive:  payroll_q3_2026.zip
  Length   Date      Time    Name
  -----   -
    506   2026-08-06  12:06   payroll_q3_2026.csv
    349   2026-08-06  12:06   MEMO_INTERNAL.txt
     46   2026-08-06  12:06   README.txt
  -----
    901
      3 files
```

Gambar 9. Hasil Memeriksa File ZIP

Kemudian archive diekstrak menggunakan:

```
unzip payroll_q3_2026.zip
```

Hasil ekstraksi:

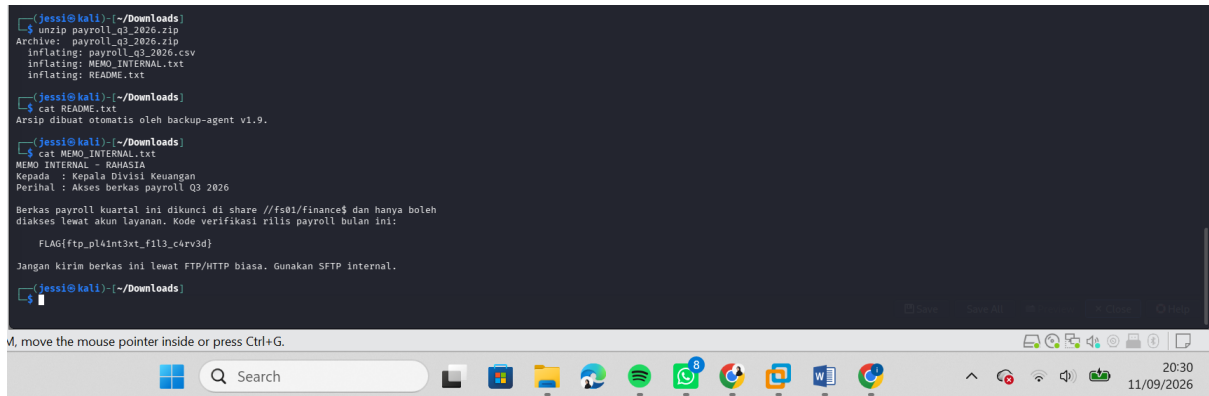
```
inflating: payroll_q3_2026.csv
inflating: MEMO_INTERNAL.txt
inflating: README.txt
```

Dari ketiga file tersebut, MEMO_INTERNAL.txt kemudian diperiksa karena berisi informasi yang berkaitan dengan aktivitas transfer file:

```
cat MEMO_INTERNAL.txt
```

Pada bagian akhir file ditemukan kode:

```
FLAG{ftp_pl41nt3xt_f1l3_c4rv3d}
```



Gambar 10. Hasil Mengekstrak Isi File ZIP

5. Hasil Analisis

Indikator	Hasil
Workstation	192.168.30.44
FTP Server	192.168.30.9
FTP Server Software	vsFTPD 3.0.5
FTP User	finops
FTP Password	FinOps#2026
Directory	/srv/finance/payroll
File yang ditransfer	payroll_q3_2026.zip
Ukuran file	930 bytes
TCP Stream	2
File hasil ekstraksi	MEMO_INTERNAL.txt
Flag	FLAG{ftp_pl41nt3xt_f1l3_c4rv3d}

6. Flag

```
FLAG{ftp_pl41nt3xt_f1l3_c4rv3d}
```

7. Conclusion

Berdasarkan analisis ftp-session.pcapng, ditemukan komunikasi FTP antara workstation 192.168.30.44 dan FTP server 192.168.30.9. Proses login menggunakan akun finops dapat terlihat pada packet capture karena FTP mengirimkan informasi autentikasi dalam bentuk plaintext.

Setelah login, client mengakses direktori /srv/finance/payroll dan meminta file payroll_q3_2026.zip menggunakan command RETR. File tersebut ditemukan pada **TCP stream 2** dengan ukuran **930 bytes**, kemudian berhasil diekspor dari packet capture dan diekstrak.

Dari `MEMO_INTERNAL.txt` yang terdapat di dalam archive ditemukan flag:

```
FLAG{ftp_pl41nt3xt_f113_c4rv3d}
```

Hasil analisis menunjukkan bahwa penggunaan FTP tanpa enkripsi memiliki risiko keamanan karena credential dan data yang ditransfer dapat terlihat pada packet capture. Untuk transfer file yang bersifat sensitif, penggunaan protokol terenkripsi seperti SFTP lebih disarankan.

Ping

1. Objective

Menganalisis traffic ICMP pada file capture untuk mengidentifikasi aktivitas yang tidak biasa pada penggunaan ICMP Echo Request, khususnya komunikasi yang dilakukan secara berulang ke alamat publik serta memeriksa apakah terdapat data yang disisipkan pada payload ICMP.

2. Target

- **Artifact:** icmp-capture.pcapng
- **Protocol:** ICMP
- **Normal traffic:** 172.16.5.11 → 8.8.8.8
- **Suspicious traffic:** 172.16.5.30 → 203.0.113.77

3. Tools

- Kali Linux
- Wireshark untuk menganalisis network traffic dan file PCAP
- TShark untuk menganalisis traffic DNS dan mengambil field dari packet capture
- xxd untuk mengubah data hexadecimal menjadi string/data asli
- base64 untuk melakukan decoding data Base64

4. Langkah Pengerjaan

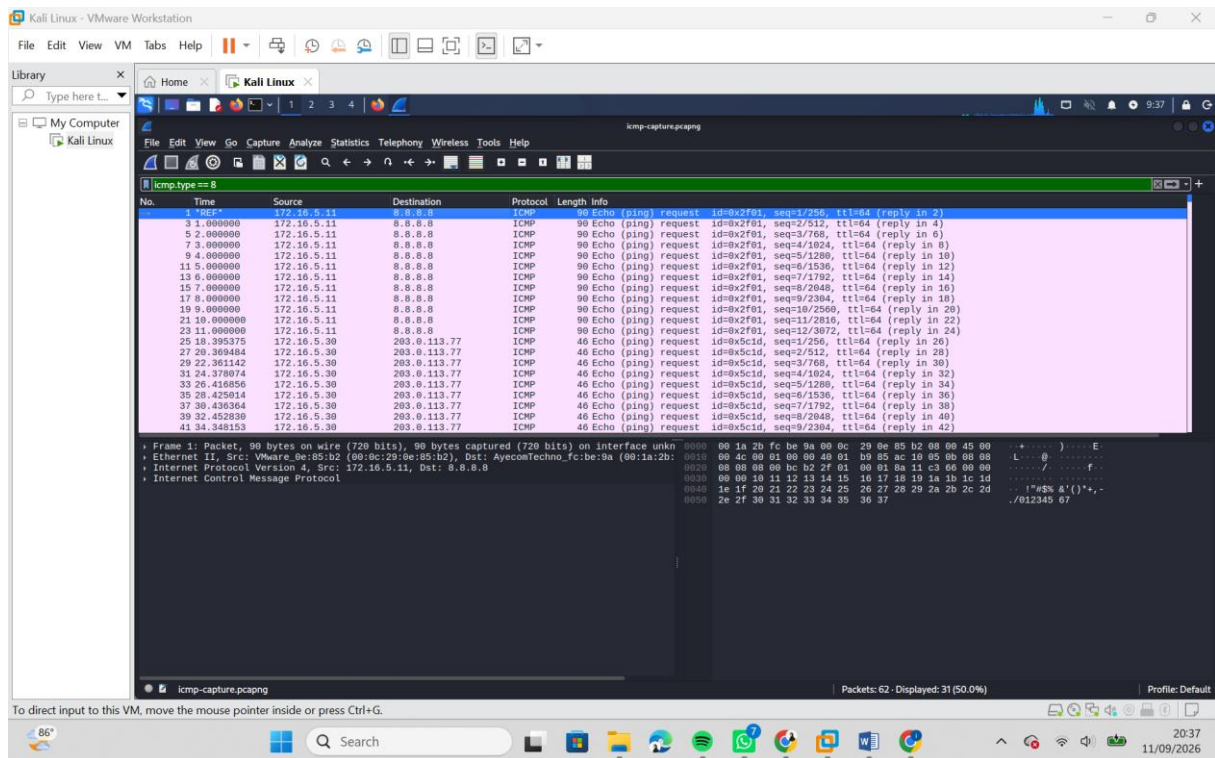
4.1 Memeriksa Traffic ICMP

File icmp-capture.pcapng dibuka menggunakan Wireshark untuk melihat traffic yang terdapat pada packet capture.

Untuk menampilkan paket **ICMP Echo Request**, digunakan filter:

```
icmp.type == 8
```

ICMP Type 8 merupakan jenis paket **Echo Request** yang umum digunakan dalam aktivitas ping.



Gambar 1. Hasil Filter ICMP Echo Request

4.2 Mengidentifikasi Traffic yang Mencurigakan

Setelah filter diterapkan, ditemukan beberapa komunikasi ICMP.

Pada awal capture terdapat komunikasi:

172.16.5.11 → 8.8.8.8

Traffic tersebut terlihat seperti aktivitas ping biasa karena dikirim secara berulang menuju 8.8.8.8.

Selain itu, ditemukan komunikasi lain:

172.16.5.30 → 203.0.113.77

Traffic ini diperiksa lebih lanjut karena memiliki pola komunikasi yang berbeda dari traffic ping sebelumnya.

4.3 Memeriksa Payload ICMP

Selanjutnya dilakukan pemeriksaan terhadap payload ICMP yang berasal dari 172.16.5.30 menuju 203.0.113.77.

Command yang digunakan:

```
tshark -r ~/Downloads/icmp-capture.pcapng \
-Y "ip.src == 172.16.5.30 && ip.dst == 203.0.113.77 && icmp.type == 8" \
```

```
-T fields -e data
```

Dari hasil tersebut ditemukan beberapa potongan data dalam format hexadecimal:

```
526b7842
52337378
59323177
58334130
65577777
4e475266
597a4232
4d334a30
58324e6f
4e473575
4d327839
```

Setiap paket membawa potongan data yang berbeda. Pola tersebut menunjukkan bahwa payload ICMP perlu diperiksa lebih lanjut karena tidak hanya berisi data ping biasa.

```
(jessi@kali)~$ tshark -r ~/Downloads/icmp-capture.pcapng -Y "icmp.type == 8" -T fields -e frame.number -e ip.src -e ip.dst -e data
1  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
2  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
3  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
4  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
5  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
6  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
7  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
8  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
9  172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
10 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
11 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
12 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
13 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
14 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
15 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
16 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
17 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
18 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
19 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
20 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
21 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
22 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
23 172.16.5.11  8.8.8.8  8a11c3660000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
24 172.16.5.30  203.0.113.77  526b7842
25 172.16.5.30  203.0.113.77  52337378
26 172.16.5.30  203.0.113.77  59323177
27 172.16.5.30  203.0.113.77  58334130
28 172.16.5.30  203.0.113.77  65577777
29 172.16.5.30  203.0.113.77  4e475266
30 172.16.5.30  203.0.113.77  597a4232
31 172.16.5.30  203.0.113.77  4d334a30
32 172.16.5.30  203.0.113.77  58324e6f
33 172.16.5.30  203.0.113.77  4e473575
34 172.16.5.30  203.0.113.77  4d327839
35 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
36 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
37 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
38 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
39 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
40 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
41 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
42 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
43 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
44 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
45 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
46 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
47 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
48 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
49 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
50 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
51 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
52 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
53 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
54 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
55 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
56 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
57 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
58 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
59 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
60 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
61 172.16.5.11  8.8.8.8  9c40d21b0000000010112131415161718191ab1c1d1e1f202122232425262728292ab2c2d2e2f3031323334353637
```

Gambar 3. Hasil Ekstraksi Payload ICMP dalam Format Hexadecimal

4.4 Menggabungkan Payload

Karena payload terbagi menjadi beberapa bagian, seluruh data kemudian digabungkan sesuai urutan paket dan dikonversi dari hexadecimal.

Command yang digunakan:

```
tshark -r ~/Downloads/icmp-capture.pcapng \
-Y "ip.src == 172.16.5.30 && ip.dst == 203.0.113.77 && icmp.type == 8" \
-T fields -e data | tr -d '\n' | xxd -r -p
```

Hasil:

```
RkxBR3sxY21wX3A0eWwwNGRfYzB2M3J0X2NoNG5uM2x9
```

Hasil tersebut menunjukkan bahwa potongan hexadecimal berhasil dikonversi menjadi sebuah string.

```
(jessi@kali)-[~]
$ tshark -r ~/Downloads/icmp-capture.pcapng \
-Y "ip.src == 172.16.5.30 && ip.dst == 203.0.113.77 && icmp.type == 8" \
-T fields -e data | tr -d '\n' | xxd -r -p
RkxBR3sxY21wX3A0eWwwNGRfYzB2M3J0X2NoNG5uM2x9
```

Gambar 4. Hasil Penggabungan dan Konversi Payload

4.5 Melakukan Decode Base64

String yang diperoleh memiliki pola Base64, sehingga dilakukan proses decoding menggunakan:

```
echo "RkxBR3sxY21wX3A0eWwwNGRfYzB2M3J0X2NoNG5uM2x9" | base64 -d
```

Hasil decoding:

```
FLAG{1cmp_p4yl04d_c0v3rt_ch4nn3l}
```

```
(jessi@kali)-[~]
$ echo "RkxBR3sxY21wX3A0eWwwNGRfYzB2M3J0X2NoNG5uM2x9" | base64 -d
FLAG{1cmp_p4yl04d_c0v3rt_ch4nn3l}
```

Gambar 5. Hasil Decode Base64 dan Flag

5. Hasil Analisis

Indikator	Hasil
Artifact	icmp-capture.pcapng
Protocol	ICMP
ICMP Type	8 – Echo Request
Normal Traffic	172.16.5.11 → 8.8.8.8
Suspicious Source	172.16.5.30
Suspicious Destination	203.0.113.77
Payload	Potongan data hexadecimal
Encoding	Base64
Hasil Decode	FLAG{1cmp_p4yl04d_c0v3rt_ch4nn3l}
Indikasi	Data dibawa melalui payload ICMP

7. Flag

```
FLAG{1cmp_p4yl04d_c0v3rt_ch4nn3l}
```

8. Conclusion

Berdasarkan analisis icmp-capture.pcapng, ditemukan traffic ICMP dari 172.16.5.30 menuju 203.0.113.77 yang membawa potongan data pada payload **Echo Request**.

Payload dari beberapa paket tersebut diekstrak dalam format hexadecimal, kemudian digabungkan dan dikonversi menjadi string Base64. Setelah dilakukan decoding, ditemukan flag:

```
FLAG{1cmp_p4yl04d_c0v3rt_ch4nn3l}
```

Dari hasil pengujian dapat disimpulkan bahwa ICMP tidak hanya dapat digunakan untuk aktivitas ping, tetapi juga dapat disalahgunakan sebagai **covert channel** untuk membawa data di dalam payload ICMP. Oleh karena itu, traffic ICMP yang diizinkan pada jaringan tetap perlu dimonitor untuk mendeteksi pola komunikasi yang tidak biasa.